```
#define _CRT_SECURE_NO_WARNINGS

#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <iostream>
#include <string>
#include <optional>
#include <vector>
#include <algorithm>
#include <random>
#include <chrono>
#include <sstream>
#include <cstring>
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <memory>
#include <unordered_map>
#include <fstream>
#include <iomanip>
#include <cctype>

#ifdef _WIN32
#include <windows.h>
#include <commdlg.h>
#endif

// ---------- Константы ----------
const int WINDOW_WIDTH = 1280;
const int WINDOW_HEIGHT = 720;
const float PANEL_WIDTH = 500.f;
const float PANEL_HEIGHT = 50.f;
const sf::Color PANEL_COLOR = sf::Color(80,
80, 80, 220);
const sf::Color TEXT_COLOR =
sf::Color::Black;
const sf::Color TEXT_SELECTED_COLOR =
sf::Color::Yellow;
const sf::Color TEXT_CORRECT_COLOR =
sf::Color::Green;
const sf::Color TEXT_WRONG_COLOR =
sf::Color::Red;

// ---------- Пути к ресурсам ----------
const std::string FONT_PATH =
"assets/arial.ttf";
const std::string BACKGROUND_PATH =
"assets/backgrounds/allback.png";
const std::string GAME_BG_PATH =
"assets/backgrounds/ex.png";
const std::string BUTTON_PATH =
"assets/buttons/button.png";
const std::string ALL_BUTTON_PATH =
"assets/buttons/all.png";
const std::string VOLUME_ON_PATH =
"assets/buttons/volumeon.png";
const std::string VOLUME_OFF_PATH =
"assets/buttons/volumeoff.png";
const std::string BUTTON_FROM_ALL_PATH =
"assets/buttons/buttonfromall.png";
const std::string CRYSTAL_LINES_PATH =
"assets/crystal/linesofcrysta.png";

const std::string MUSIC_PATH1 =
"assets/sounds/mus1.mp3";
const std::string MUSIC_PATH2 =
"assets/sounds/var2.mp3";
const std::string MUSIC_PATH3 =
"assets/sounds/var3.mp3";
const std::string TEXT_SOUND_PATH =
"assets/sounds/text.mp3";
const std::string RESULT_SOUND_PATH =
"assets/sounds/result.mp3";

const std::string CLICK_SOUND_PATH =
"assets/sounds/click.ogg";
const std::string GOOD_SOUND_PATH =
"assets/sounds/goodanswer.mp3";
const std::string BAD_SOUND_PATH =
"assets/sounds/badanswer.mp3";
const std::string RESULT_END_SOUND_PATH =
"assets/sounds/resultend.mp3";

const std::string CRYSTAL_FRAGMENT_PREFIX =
"assets/crystal/crs";

// ---------- НАСТРОЙКА ПОЛОЖЕНИЯ КРИСТАЛЛА
----------
const sf::Vector2f GLOBAL_OFFSET = { 0.f,
0.f };
const sf::Vector2f fragmentOffsets[10] = {
    {50.f, 104.f},   // 1
    {117.f,82.f},    // 2
    {84.f, -3.f},    // 3
    {160.f, -3.f},   // 4
    {70.f,-10.f},    // 5
    {-23.f, 44.f},   // 6
    {-83.f, 14.f},   // 7
    {-2.f, 22.f},    // 8
    {-40.f, -55.f},  // 9
    {30.f,-20.f}     // 10
};
const sf::Vector2f fragmentPositionsBase[10]
= {
    {651, 171}, {483, 228}, {595, 421},
{425, 421}, {624, 435},
    {829, 317}, {952, 380}, {783, 354},
{866, 520}, {613, 611}
};

// ---------- Вспомогательные функции ------
----
static sf::String utf8_to_sfstring(const
std::string& utf8) {
    return
sf::String::fromUtf8(utf8.begin(),
utf8.end());
}
static bool containsPoint(const
sf::FloatRect& rect, const sf::Vector2f&
point) {
    return rect.contains(point);
}
void centerOrigin(sf::Sprite& sprite) {
    sf::FloatRect bounds =
sprite.getLocalBounds();
    sprite.setOrigin({ bounds.position.x +
bounds.size.x / 2.f,
                       bounds.position.y +
bounds.size.y / 2.f });
}
void centerTextOnButton(sf::Text& text,
const sf::Sprite& button) {
    sf::FloatRect tb =
text.getLocalBounds();
    sf::FloatRect sb =
button.getGlobalBounds();
    text.setPosition({ sb.position.x +
(sb.size.x - tb.size.x) / 2.f,
                       sb.position.y +
(sb.size.y - tb.size.y) / 2.f -
tb.position.y });
}

// Удаление всех пробелов и управляющих
символов
std::string normalizeString(const
std::string& s) {
    std::string result;
    for (char c : s) {
        if
(!std::isspace(static_cast<unsigned
char>(c))) {
            result += c;
        }
    }
    return result;
}
```

```
// Удаление всех пробелов, табуляций,
переносов строк (для сравнения кода)
std::string normalizeCode(const std::string&
s) {
    std::string result;
    for (char c : s) {
        if
(!std::isspace(static_cast<unsigned
char>(c))) {
            result += c;
        }
    }
    return result;
}

std::string wrapText(const std::string&
text, sf::Font& font, unsigned int charSize,
float maxWidth) {
    std::string result;
    std::string currentLine;
    std::istringstream words(text);
    std::string word;
    while (words >> word) {
        std::string testLine =
currentLine.empty() ? word : currentLine + "
" + word;
        sf::Text test(font,
utf8_to_sfstring(testLine), charSize);
        if (test.getLocalBounds().size.x >
maxWidth) {
            if (!currentLine.empty()) {
                result += currentLine +
"\n";
                currentLine = word;
            }
            else {
                result += word + "\n";
                currentLine = "";
            }
        }
        else {
            currentLine = testLine;
        }
    }
    if (!currentLine.empty()) result +=
currentLine;
    return result;
}

// Проверка попадания в спрайт с уменьшенной
областью (padding)
bool isPointInSpriteWithPadding(const
sf::Sprite& sprite, const sf::Vector2f&
point, float padding = 10.f) {
    sf::FloatRect bounds =
sprite.getGlobalBounds();
    bounds.position.x += padding;
    bounds.position.y += padding;
    bounds.size.x -= 2 * padding;
    bounds.size.y -= 2 * padding;
    if (bounds.size.x < 0 || bounds.size.y <
0) return false;
    return bounds.contains(point);
}

// ---------- Типы вопросов ----------
enum class QuestionType { CHOICE, FILL_CODE,
FIND_ERROR, ORDER };
struct OrderLine { std::string text; int
correctOrder; int currentOrder; };
struct Question {
    int id;
    QuestionType type;
    std::string questionText;
    std::vector<std::string> options;
    std::vector<int> correctOptions;
    std::string codeSnippet;
    std::string expectedFill;
    std::string brokenCode;
    std::string fixedCode;
    std::vector<OrderLine> lines;
    std::string hint;   // подсказка для
FIND_ERROR

    Question(int id, QuestionType type,
const std::string& questionText,
        const std::vector<std::string>&
options = {}, const std::vector<int>&
correctOptions = {},
        const std::string& codeSnippet = "",
const std::string& expectedFill = "",
        const std::string& brokenCode = "",
const std::string& fixedCode = "",
        const std::vector<OrderLine>& lines
= {}, const std::string& hint = "")
        : id(id), type(type),
questionText(questionText),
options(options),
        correctOptions(correctOptions),
codeSnippet(codeSnippet),
expectedFill(expectedFill),
        brokenCode(brokenCode),
fixedCode(fixedCode), lines(lines),
hint(hint) {
    }
};

// ---------- Пул вопросов ----------
std::vector<Question> questionPool;

void initQuestionPool() {
    questionPool.clear();

    // ---- Теория (CHOICE) ----
    questionPool.emplace_back(1,
QuestionType::CHOICE,
        "В чем принципиальное отличие между
статической и динамической памятью?",
        std::vector<std::string>{
        "А) Статическая память быстрее, но
её нельзя менять; динамическая медленнее, но
гибче.",
            "Б) Статическая память очищается
сама при выходе из функции; динамическую
нужно освобождать вручную.",
            "В) Динамическая память всегда
выделяется при компиляции, а статическая —
при запуске.",
            "Г) Между ними нет никакой
разницы."
    }, std::vector<int>{1});

    questionPool.emplace_back(2,
QuestionType::CHOICE,
        "Почему в C++ принята индексация
массивов с 0, а не с 1?",
        std::vector<std::string>{
        "А) Это историческая ошибка первых
разработчиков языка.",
            "Б) Потому что индекс — это
смещение от начала массива в памяти.",
            "В) Индексация с 1 замедляет
работу процессора.",
            "Г) Так требует стандарт
безопасности данных."
    }, std::vector<int>{1});

    questionPool.emplace_back(3,
QuestionType::CHOICE,
        "Что такое «утечка памяти» при
работе с динамическими массивами?",
        std::vector<std::string>{
        "А) Ошибка при компиляции кода.",
            "Б) Потеря доступа к выделенной
памяти без её последующего освобождения
через delete[].",
            "В) Слишком быстрое заполнение
массива данными.",
            "Г) Запись данных за границы
массива."
```

```
    }, std::vector<int>{1});

    questionPool.emplace_back(4,
QuestionType::CHOICE,
        "Можно ли изменить размер
статического массива int arr[10]; во время
выполнения программы?",
        std::vector<std::string>{
        "А) Да, используя метод resize().",
            "Б) Да, если массив объявлен как
глобальный.",
            "В) Нет, размер статического
массива фиксируется на этапе компиляции.",
            "Г) Да, если использовать
оператор new."
    }, std::vector<int>{2});

    questionPool.emplace_back(5,
QuestionType::CHOICE,
        "Что из себя представляет двумерный
массив int a[3][3] с точки зрения памяти?",
        std::vector<std::string>{
        "А) Список независимых указателей,
разбросанных по разным частям памяти.",
            "Б) Непрерывный блок памяти,
который логически воспринимается как
матрица.",
            "В) Несколько объектов vector,
объединенных в один класс.",
            "Г) Таблицу, состоящую из
объектов типа string."
    }, std::vector<int>{1});

    questionPool.emplace_back(6,
QuestionType::CHOICE,
        "Какой индекс у последнего элемента
массива, если размер массива равен 15?",
        std::vector<std::string>{"А) 14",
"Б) 15", "В) 16", "Г) 0"},
std::vector<int>{0});

    questionPool.emplace_back(7,
QuestionType::CHOICE,
        "Какая запись является корректным
объявлением целочисленного массива из 20
элементов?",
        std::vector<std::string>{"А) int
arr(20);", "Б) array arr[20];", "В) int
arr[20];", "Г) int arr{20};"},
std::vector<int>{2});

    questionPool.emplace_back(8,
QuestionType::CHOICE,
        "Что произойдет при попытке
обращения к arr[10] в массиве, объявленном
как int arr[10];?",
        std::vector<std::string>{"А)
Автоматическое расширение массива.", "Б)
Программа вернет значение 0.", "В) Возникнет
ошибка компиляции.", "Г) Произойдет выход за
границы массива (неопределенное
поведение)."}, std::vector<int>{3});

    questionPool.emplace_back(9,
QuestionType::CHOICE,
        "В какой области памяти размещаются
статические массивы, объявленные внутри
функции?",
        std::vector<std::string>{"А) В
статической памяти.", "Б) В стековой памяти
(автоматическая память).", "В) В куче
(heap).", "Г) В регистрах процессора."},
std::vector<int>{1});

    questionPool.emplace_back(10,
QuestionType::CHOICE,
        "Какое ключевое слово используется
для выделения памяти под динамический
массив?",
        std::vector<std::string>{"А)
malloc", "Б) alloc", "В) new", "Г) create"},
std::vector<int>{2});

    questionPool.emplace_back(11,
QuestionType::CHOICE,
        "Как правильно освободить память,
выделенную под динамический массив int* ptr
= new int[5];?",
        std::vector<std::string>{"А) delete
ptr;", "Б) free(ptr);", "В) delete[] ptr;",
"Г) erase(ptr);"}, std::vector<int>{2});

    questionPool.emplace_back(12,
QuestionType::CHOICE,
        "Как выглядит правильный синтаксис
обращения к элементу на пересечении 2-й
строки и 3-го столбца в двумерном массиве
matrix?",
        std::vector<std::string>{"А)
matrix[2, 3]", "Б) matrix[1][2]", "В)
matrix[2][3]", "Г) matrix(2)(3)"},
std::vector<int>{1});

    questionPool.emplace_back(13,
QuestionType::CHOICE,
        "Какое значение по умолчанию у
элементов статического массива типа int,
если он объявлен глобально?",
        std::vector<std::string>{"А)
Случайное число.", "Б) 0.", "В) -1.", "Г)
Значение не определено."},
std::vector<int>{1});

    questionPool.emplace_back(14,
QuestionType::CHOICE,
        "Какую библиотеку необходимо
подключить для использования динамического
контейнера vector?",
        std::vector<std::string>{"А)
<array>", "Б) <vector>", "В) <list>", "Г)
<iostream>"}, std::vector<int>{1});

    questionPool.emplace_back(15,
QuestionType::CHOICE,
        "Для чего в работе с массивами
используют функцию srand(time(0));?",
        std::vector<std::string>{"А) Для
очистки памяти массива.", "Б) Для сортировки
массива.", "В) Для инициализации генератора
случайных чисел.", "Г) Для изменения размера
массива."}, std::vector<int>{2});

    // ---- Задания на порядок (ORDER) ----
    auto addOrderQuestion = [&](int id,
const std::string& desc, const
std::vector<std::string>& codeLines) {
        std::vector<OrderLine> lines;
        for (size_t i = 0; i <
codeLines.size(); ++i) {
            lines.push_back({ codeLines[i],
static_cast<int>(i), static_cast<int>(i) });
        }
        questionPool.emplace_back(id,
QuestionType::ORDER, desc,
std::vector<std::string>{},
std::vector<int>{}, "", "", "", "", lines);
        };

    addOrderQuestion(16, "Расставьте строки
кода в правильном порядке для вычисления
суммы элементов массива:",
        {
            "int arr[] = {10, 20, 30, 40,
50};",
            "int sum = 0;",
            "for (int i = 0; i < 5; i++) {",
            "    sum += arr[i];",
            "}",
            "cout << \"Сумма: \" << sum;"
```

```
        });

    addOrderQuestion(17, "Расставьте строки
кода в правильном порядке для создания и
удаления динамического массива:",
        {
            "int n = 5;",
            "int* ptr = new int[n];",
            "delete[] ptr;",
            "ptr = nullptr;"
        });

    addOrderQuestion(18, "Расставьте строки
кода в правильном порядке для ввода
элементов двумерного массива (матрицы):",
        {
            "int mat[2][2];",
            "for (int i = 0; i < 2; i++) {",
            "    for (int j = 0; j < 2; j++)
{",
            "        cin >> mat[i][j];",
            "    }",
            "}"
        });

    addOrderQuestion(19, "Расставьте строки
кода в правильном порядке для поиска
максимального элемента в массиве:",
        {
            "int arr[] = {5, 2, 9, 1, 7};",
            "int maxVal = arr[0];",
            "for (int i = 1; i < 5; i++) {",
            "    if (arr[i] > maxVal) maxVal
= arr[i];",
            "}"
        });

    addOrderQuestion(20, "Расставьте строки
кода в правильном порядке для копирования
одного массива в другой:",
        {
            "int src[] = {1, 2, 3};",
            "int dest[3];",
            "for (int i = 0; i < 3; i++) {",
            "    dest[i] = src[i];",
            "}"
        });

    addOrderQuestion(21, "Расставьте строки
кода в правильном порядке для вывода
элементов массива в обратном порядке:",
        {
            "int arr[] = {10, 20, 30};",
            "for (int i = 2; i >= 0; i--)
{",
            "    cout << arr[i] << \" \";",
            "}"
        });

    addOrderQuestion(22, "Расставьте строки
кода в правильном порядке для инициализации
вектора и добавления элемента:",
        {
            "#include <vector>",
            "using namespace std;",
            "vector<int> v;",
            "v.push_back(100);"
        });

    addOrderQuestion(23, "Расставьте строки
кода в правильном порядке для проверки
наличия четных чисел в массиве:",
        {
            "int arr[] = {1, 3, 5, 4};",
            "for (int i = 0; i < 4; i++) {",
            "    if (arr[i] % 2 == 0) cout
<< arr[i];",
            "}"
        });

    addOrderQuestion(24, "Расставьте строки
кода в правильном порядке для обнуления
двумерного массива:",
        {
            "int mat[3][3];",
            "for (int i = 0; i < 3; i++) {",
            "    for (int j = 0; j < 3; j++)
{",
            "        mat[i][j] = 0;",
            "    }",
            "}"
        });

    // ---- ЗАДАНИЯ НА ИСПРАВЛЕНИЕ ОШИБОК
(FIND_ERROR) с подсказками ----
    auto addFindError = [&](int id, const
std::string& desc, const std::string&
broken, const std::string& fixed, const
std::string& hint) {
        questionPool.emplace_back(id,
QuestionType::FIND_ERROR, desc,
            std::vector<std::string>{},
std::vector<int>{}, "", "", broken, fixed,
            std::vector<OrderLine>{}, hint);
        };

    addFindError(25, "Исправьте ошибку в
коде (выход за границы массива):",
        "int arr[10];\nfor (int i = 1; i <=
10; i++) {\n    arr[i] = i * 2;\n    cout <<
arr[i] << \" \";\n}\ncout << endl;",
        "int arr[10];\nfor (int i = 0; i <
10; i++) {\n    arr[i] = i * 2;\n    cout <<
arr[i] << \" \";\n}\ncout << endl;",
        "В C++ индексация массивов
начинается с 0. Для массива размером 10
допустимые индексы — от 0 до 9. Условие i <=
10 приводит к выходу за пределы памяти
(UB).");

    addFindError(26, "Исправьте ошибку
освобождения динамической памяти:",
        "int n = 20;\nint* dynamicArray =
new int[n];\nfor (int i = 0; i < n; i++) {\n
dynamicArray[i] = i + 5;\n    cout <<
dynamicArray[i] << \" \";\n}\ndelete
dynamicArray;",
        "int n = 20;\nint* dynamicArray =
new int[n];\nfor (int i = 0; i < n; i++) {\n
dynamicArray[i] = i + 5;\n    cout <<
dynamicArray[i] << \" \";\n}\ndelete[]
dynamicArray;",
        "Память, выделенная через new[] (для
массивов), должна освобождаться строго с
помощью delete[]. Одиночный delete удалит
только первый элемент, вызвав утечку
памяти.");

    addFindError(27, "Исправьте ошибку
объявления статического массива с переменным
размером:",
        "int rows = 3;\nint cols = 4;\nint
matrix[rows][cols];\nfor (int i = 0; i <
rows; i++) {\n    for (int j = 0; j < cols;
j++) {\n        matrix[i][j] = 1;\n
}\n}",
        "const int rows = 3;\nconst int cols
= 4;\nint matrix[rows][cols];\nfor (int i =
0; i < rows; i++) {\n    for (int j = 0; j <
cols; j++) {\n        matrix[i][j] = 1;\n
}\n}",
        "Размер статического массива должен
быть известен на этапе компиляции. Обычная
переменная может измениться, поэтому размер
должен быть задан через const.");

    addFindError(28, "Исправьте ошибку при
добавлении элементов в вектор:",
```

```
        "vector<int> scores;\nfor (int i =
0; i < 5; i++) {\n    scores[i] = i * 10;\n
cout << scores[i] << \"\\n\";\n}",
        "vector<int> scores;\nfor (int i =
0; i < 5; i++) {\n    scores.push_back(i *
10);\n    cout << scores[i] << \"\\n\";\n}",
        "Вектор изначально пуст. Обращение
по индексу v[i] допустимо только для
существующих элементов. Для добавления
используйте push_back().");

    addFindError(29, "Исправьте ошибку с
неинициализированным указателем:",
        "int total = 500;\nint* ptr;\nint
val;\n*ptr = total;\nval = *ptr;\ncout <<
val;",
        "int total = 500;\nint* ptr =
&total;\nint val;\n*ptr = total;\nval =
*ptr;\ncout << val;",
        "Указатель должен быть
инициализирован адресом существующей
переменной перед разыменованием. Иначе он
указывает в случайную область памяти.");

    addFindError(30, "Исправьте ошибку с
вызовом srand внутри цикла:",
        "int randomNumbers[5];\nfor (int i =
0; i < 5; i++) {\n    srand(time(NULL));\n
randomNumbers[i] = rand() % 10;\n    cout <<
randomNumbers[i] << \" \";\n}",
        "int
randomNumbers[5];\nsrand(time(NULL));\nfor
(int i = 0; i < 5; i++) {\n
randomNumbers[i] = rand() % 10;\n    cout <<
randomNumbers[i] << \" \";\n}",
        "srand() должна вызываться один раз
в начале программы, иначе при быстрых
итерациях числа будут повторяться из-за
одинакового времени.");

    addFindError(31, "Исправьте ошибку
поиска минимума (неверная инициализация):",
        "int arr[5] = {-5, -10, -3, -1, -
8};\nint minVal = 0;\nfor (int i = 0; i < 5;
i++) {\n    if (arr[i] < minVal) {\n
minVal = arr[i];\n    }\n}\ncout <<
minVal;",
        "int arr[5] = {-5, -10, -3, -1, -
8};\nint minVal = arr[0];\nfor (int i = 1; i
< 5; i++) {\n    if (arr[i] < minVal) {\n
minVal = arr[i];\n    }\n}\ncout <<
minVal;",
        "Инициализация minVal = 0 неверна,
если все числа отрицательные. Правильно
брать первый элемент массива как начальное
значение.");

    addFindError(32, "Исправьте ошибку с
итератором конца вектора:",
        "vector<int> v(10,
5);\nvector<int>::iterator start =
v.begin();\nvector<int>::iterator end =
v.end() + 1;\nvector<int> v2(start, end);",
        "vector<int> v(10,
5);\nvector<int>::iterator start =
v.begin();\nvector<int>::iterator end =
v.end();\nvector<int> v2(start, end);",
        "Итератор end() указывает на элемент
после последнего. Прибавление 1 выходит за
пределы контейнера. Используйте просто
end().");

    addFindError(33, "Исправьте ошибку
инициализации двумерного массива и выхода за
границы:",
        "int matrix[3][3] = {\n    {1, 2,
3},\n    {4, 5},\n    {7, 8, 9,
10}\n};\ncout << matrix[2][3];",
        "int matrix[3][3] = {\n    {1, 2,
3},\n    {4, 5, 0},\n    {7, 8, 9}\n};\ncout
<< matrix[2][2];",
        "Вторая строка содержит только 2
элемента, а третья – 4, что превышает
размер. Кроме того, индекс [2][3] выходит за
границы (максимум [2][2]).");

    addFindError(34, "Исправьте ошибку
двойного удаления и утечки памяти:",
        "int* a = new int(10);\nint* b = new
int(20);\na = b;\ndelete a;\ndelete b;",
        "int* a = new int(10);\nint* b = new
int(20);\ndelete a;\ndelete b;",
        "После a = b оба указателя указывают
на одну область памяти. Удаление дважды
приводит к UB. Удаляйте каждый объект один
раз.");

    addFindError(35, "Исправьте ошибку
перепутанных границ циклов в матрице:",
        "int matrix[4][5];\nfor(int i = 0; i
< 5; i++) {\n    for(int j = 0; j < 4; j++)
{\n        matrix[i][j] = i + j;\n    }\n}",
        "int matrix[4][5];\nfor(int i = 0; i
< 4; i++) {\n    for(int j = 0; j < 5; j++)
{\n        matrix[i][j] = i + j;\n    }\n}",
        "Перепутаны границы строк (4) и
столбцов (5). Внешний цикл должен идти до 4,
внутренний до 5.");

    addFindError(36, "Исправьте ошибку
использования insert в векторе:",
        "vector<int> nums = {10, 20,
30};\nnums.insert(1, 15);\nfor(int n : nums)
{\n    cout << n << \" \";\n}",
        "vector<int> nums = {10, 20,
30};\nnums.insert(nums.begin() + 1,
15);\nfor(int n : nums) {\n    cout << n <<
\" \";\n}",
        "Метод insert принимает итератор
позиции и значение. В первом аргументе
должно быть nums.begin() + 1, а не просто
число 1.");

    addFindError(37, "Исправьте ошибку
присваивания указателю целого числа:",
        "int a = 15;\nint b = 25;\nint* p1 =
&a;\nint* p2 = &b;\np1 = *p2;\ncout <<
*p1;",
        "int a = 15;\nint b = 25;\nint* p1 =
&a;\nint* p2 = &b;\np1 = p2;\ncout << *p1;",
        "Присваивание p1 = *p2 пытается
присвоить указателю значение int, что
недопустимо. Нужно присвоить адрес: p1 =
p2.");

    addFindError(38, "Исправьте ошибку
выхода за границу массива в цикле:",
        "int arr[5] = {1, 2, 3, 4, 5};\nint
sum = 0;\nfor(int i = 0; i <= 5; i++) {\n
if(arr[i] % 2 != 0) {\n        sum +=
arr[i];\n    }\n}",
        "int arr[5] = {1, 2, 3, 4, 5};\nint
sum = 0;\nfor(int i = 0; i < 5; i++) {\n
if(arr[i] % 2 != 0) {\n        sum +=
arr[i];\n    }\n}",
        "Условие i <= 5 приводит к обращению
к arr[5], которого нет. Исправьте на i <
5.");

    addFindError(39, "Исправьте ошибку
разыменования указателя за пределами
массива:",
        "const int N = 5;\nint arr[N] = {1,
2, 3, 4, 5};\nint* p = arr;\np = p +
N;\ncout << *p;",
        "const int N = 5;\nint arr[N] = {1,
2, 3, 4, 5};\nint* p = arr;\np = p + (N -
1);\ncout << *p;",
```

```
        "Указатель p + N указывает за пределы массива. Допустимо только p + (N-1) для доступа к последнему элементу.");

    // ---- ЗАДАНИЯ НА ВОССТАНОВЛЕНИЕ ПРОПУСКОВ (FILL_CODE) ----
    auto addFillCode = [&](int id, const std::string& desc, const std::string& snippet, const std::string& expected) {
        questionPool.emplace_back(id, QuestionType::FILL_CODE, desc,
            std::vector<std::string>{}, std::vector<int>{}, snippet, expected);
        };

    addFillCode(40, "Восстановите пропуск для генерации случайного числа в диапазоне [10,20]:",
        "int arr[10];\nsrand(time(NULL));\nfor(int i = 0; i < 10; i++) {\n    arr[i] = 10 + rand() ___ (20 - 10 + 1);\n    cout << arr[i] << \" \";\n}",
        "%");

    addFillCode(41, "Восстановите пропуски для выделения и освобождения динамического массива:",
        "int n = 15;\nint* data = ___ int[n];\nfor(int i = 0; i < n; i++) {\n data[i] = i * i;\n}\ndelete___ data;",
        "new[]");

    addFillCode(42, "Восстановите знак сравнения для поиска максимума в матрице:",
        "int mat[3][3] = {{1, 2, 3}, {4, 9, 6}, {7, 8, 5}};\nint m = mat[0][0];\nfor(int i = 0; i < 3; i++) {\n    for(int j = 0; j < 3; j++) {\n        if(mat[i][j] ___ m) {\n m = mat[i][j];\n        }\n    }\n}",
        ">");

    addFillCode(43, "Восстановите метод для добавления элемента в конец вектора:",
        "vector<int> v1 = {1, 2, 3};\nvector<int> v2(v1);\nv2.____(4);\nfor(int i = 0; i < v2.size(); i++) {\n    cout << v2[i] << \" \";\n}",
        "push_back");

    addFillCode(44, "Восстановите условие цикла для обхода массива из 10 элементов:",
        "int arr[10];\nfor (int i = 0; ... ; i++) {\n    cout << arr[i];\n}",
        "i < 10");

    addFillCode(45, "Восстановите инициализацию суммы для вычисления суммы элементов:",
        "int arr[] = {1, 2, 3, 4, 5};\nint sum = ... ;\nfor (int i = 0; i < 5; i++) {\n sum += arr[i];\n}",
        "0");

    addFillCode(46, "Восстановите делитель для получения среднего арифметического (дробная часть):",
        "int arr[5] = {2, 4, 6, 8, 10};\nint sum = 0;\nfor(int i = 0; i < 5; i++) {\n sum += arr[i];\n}\nfloat avg = sum / ___;\ncout << avg;",
        "5.0");

    addFillCode(47, "Восстановите границы вложенных циклов для матрицы 2x4:",
        "int M[2][4] = {{1,1,1,1}, {2,2,2,2}};\nint total = 0;\nfor(int i = 0; i < ___; i++) {\n    for(int j = 0; j < ___; j++) {\n        total += M[i][j];\n }\n}",
        "2 и 4");

    addFillCode(48, "Восстановите выражение для вывода элемента вектора по индексу:",
        "vector<int> v = {1, 2, 3};\nfor (int i = 0; i < v.size(); i++) {\n    cout << ... << \" \";\n}",
        "v[i]");

    addFillCode(49, "Восстановите метод изменения размера вектора и получения текущего размера:",
        "vector<int> v = {1, 2};\nv.____(5, 10);\nfor(int i = 0; i < v.____(); i++) {\n cout << v[i] << \" \";\n}",
        "resize и size");

    addFillCode(50, "Восстановите размер второго измерения для инициализации матрицы:",
        "int arr[][___] = {\n    {1, 2, 3},\n    {4, 5, 6}\n};\ncout << arr[1][2];",
        "3");

    addFillCode(51, "Восстановите метод удаления последнего элемента вектора:",
        "vector<int> v = {7, 8, 9};\nv.____();\ncout << v.back();",
        "pop_back");

    addFillCode(52, "Восстановите оператор разыменования для вывода значения по указателю:",
        "int arr[3] = {10, 20, 30};\nint* p = arr;\np++;\ncout << ___p;",
        "*");

    addFillCode(53, "Восстановите условие цикла для вывода массива в обратном порядке до нулевого элемента:",
        "int n = 5;\nint* arr = new int[n];\nfor(int i = 0; i < n; i++) cin >> arr[i];\nfor(int i = n - 1; i ___ 0; i--) {\n    cout << arr[i] << \" \";\n}",
        ">=");
}

// Выбор 10 случайных вопросов из пула
std::vector<Question> selectRandomQuestions() {
    std::vector<Question> selected;
    if (questionPool.empty()) {
        questionPool.emplace_back(1, QuestionType::CHOICE,
            "Тестовый вопрос: 2+2?",
            std::vector<std::string>{"3", "4", "5", "6"}, std::vector<int>{1});
    }
    auto seed = static_cast<unsigned int>(std::chrono::steady_clock::now().time_since_epoch().count());
    std::default_random_engine rng(seed);
    std::vector<Question> poolCopy = questionPool;
    std::shuffle(poolCopy.begin(), poolCopy.end(), rng);
    int count = std::min(10, static_cast<int>(poolCopy.size()));
    selected.insert(selected.end(), poolCopy.begin(), poolCopy.begin() + count);
    return selected;
}

// ---------- Анимированный текст ----------
class AnimatedText {
private:
    sf::Text text;
    std::string fullText;
```

```
    std::size_t currentPos = 0;
    sf::Clock clock;
    float charDelay = 0.02f;
    bool finished = false;
    sf::Sound* typingSound = nullptr;
public:
    AnimatedText(sf::Font& font) :
text(font) {}
    void setString(const std::string& str) {
fullText = str; currentPos = 0; finished =
false; text.setString(""); clock.restart();
}
    void setPosition(const sf::Vector2f&
pos) { text.setPosition(pos); }
    void setCharacterSize(unsigned int size)
{ text.setCharacterSize(size); }
    void setFillColor(const sf::Color&
color) { text.setFillColor(color); }
    void update() {
        if (!finished &&
clock.getElapsedTime().asSeconds() >=
charDelay) {
            if (currentPos <
fullText.length()) {
                currentPos++;
                sf::String s =
sf::String::fromUtf8(fullText.begin(),
fullText.begin() +
static_cast<std::ptrdiff_t>(currentPos));
                text.setString(s);
                clock.restart();
                if (typingSound &&
typingSound->getStatus() !=
sf::SoundSource::Status::Playing) {
                    typingSound->play();
                }
            }
            else {
                finished = true;
                if (typingSound &&
typingSound->getStatus() ==
sf::SoundSource::Status::Playing) {
                    typingSound->stop();
                }
            }
        }
    }
    void draw(sf::RenderWindow& window) {
window.draw(text); }
    void reset() { currentPos = 0; finished
= false; text.setString("");
clock.restart(); }
    float getTextWidth() const { return
text.getGlobalBounds().size.x; }
    float getTextHeight() const { return
text.getGlobalBounds().size.y; }
    bool isFinished() const { return
finished; }
    void setTypingSound(sf::Sound* sound) {
typingSound = sound; }
    void stopSound() { if (typingSound)
typingSound->stop(); }
    void restartSound() { if (typingSound &&
!finished) { typingSound->play(); } }
};
int main() {

std::srand(static_cast<unsigned>(std::time(n
ullptr)));

    // ---- Окно с заголовком, без изменения
размера, с иконкой ----
    sf::RenderWindow window(sf::VideoMode({
WINDOW_WIDTH, WINDOW_HEIGHT }),
        L"Массивы в языке C++ - игра",
        sf::Style::Titlebar |
sf::Style::Close);
    window.setFramerateLimit(60);

    // Загрузка иконки (если файл
существует)
    sf::Image icon;
    if
(icon.loadFromFile("assets/icon.png")) {
        window.setIcon(icon);
    }

    // ---- Шрифт и текстуры ----
    auto font =
std::make_unique<sf::Font>();
    if (!font->openFromFile(FONT_PATH)) {
std::cerr << "Font error\n"; return -1; }
    // ЭКРАН ЗАГРУЗКИ (ПЛАВНЫЙ И КРАСИВЫЙ)
    sf::Clock loadingClock;
    sf::Text loadingText(*font,
utf8_to_sfstring("Загрузка..."), 26);

loadingText.setFillColor(sf::Color::White);
    loadingText.setPosition({ WINDOW_WIDTH /
2.f - loadingText.getLocalBounds().size.x /
2.f, WINDOW_HEIGHT / 2.f - 60.f });

    // Задний фон для полосы загрузки
    sf::RectangleShape barBg({ 400.f, 16.f
});
    barBg.setFillColor(sf::Color(50, 50,
50));
    barBg.setPosition({ WINDOW_WIDTH / 2.f -
200.f, WINDOW_HEIGHT / 2.f });

    // Сам прогресс-бар (зеленого цвета)
    sf::RectangleShape bar({ 0.f, 16.f });
    bar.setFillColor(sf::Color(46, 204,
113));
    bar.setPosition({ WINDOW_WIDTH / 2.f -
200.f, WINDOW_HEIGHT / 2.f });

    bool loading = true;
    while (loading && window.isOpen()) {
        while (const std::optional event =
window.pollEvent()) {
            // Проверяем нажатие клавиш (для
SFML 3.0)
           // Проверяем нажатие клавиш (для
SFML 3.0)
            if (const auto* keyPressed =
event->getIf<sf::Event::KeyPressed>()) {
                if (keyPressed->code ==
sf::Keyboard::Key::F1) {
#ifdef _WIN32
                    // ТЕСТ: Если это окно
появится, значит SFML успешно перехватил
нажатие кнопки F1
                    MessageBoxA(NULL,
"Кнопка F1 работает! Запускаю
CreateProcess...", "Отладка", MB_OK |
MB_ICONINFORMATION);

                    // 1. Строим путь
относительно запущенного файла .exe
                    char exePath[MAX_PATH];
                    GetModuleFileNameA(NULL,
exePath, MAX_PATH);
                    std::string
pathStr(exePath);
                    size_t pos =
pathStr.find_last_of("\\/");
                    if (pos !=
std::string::npos) {
                        pathStr =
pathStr.substr(0, pos);
                    }
                    std::string
fullPathFromExe = pathStr +
"\\assets\\help\\NewProject.chm";

                    // 2. Строим путь
относительно рабочей папки проекта Visual
Studio
```

```
                    std::string
fullPathFromProject =
"assets\\help\\NewProject.chm";

                    // Формируем итоговую
команду запуска
                    std::string cmd;
                    DWORD dwAttrib =
GetFileAttributesA(fullPathFromExe.c_str());
                    if (dwAttrib !=
INVALID_FILE_ATTRIBUTES && !(dwAttrib &
FILE_ATTRIBUTE_DIRECTORY)) {
                        cmd = "hh.exe \"" +
fullPathFromExe + "\"";
                    }
                    else {
                        cmd = "hh.exe \"" +
fullPathFromProject + "\"";
                    }

                    // Настройка структур
для CreateProcessA
                    STARTUPINFOA si;
                    PROCESS_INFORMATION pi;
                    ZeroMemory(&si,
sizeof(si));
                    si.cb = sizeof(si);
                    ZeroMemory(&pi,
sizeof(pi));

                    // ВАЖНО: CreateProcessA
требует изменяемый буфер (не const char*),
                    // поэтому копируем
строку в вектор/массив символов.
                    std::vector<char>
cmdBuffer(cmd.begin(), cmd.end());

cmdBuffer.push_back('\0');

                    // Запуск процесса
                    if (CreateProcessA(
                        NULL,              //
Имя модуля (используем командную строку)
                        cmdBuffer.data(), //
Командная строка (изменяемый буфер)
                        NULL,              //
Атрибуты безопасности процесса
                        NULL,              //
Атрибуты безопасности потока
                        FALSE,             //
Наследование дескрипторов
                        0,                 //
Флаги создания
                        NULL,              //
Блок переменных окружения
                        NULL,              //
Текущий диск/каталог
                        &si,               //
Указатель на структуру STARTUPINFO
                        &pi                //
Указатель на структуру PROCESS_INFORMATION
                    )) {
                        // Обязательно
закрываем дескрипторы, чтобы избежать утечки
ресурсов в игре

CloseHandle(pi.hProcess);

CloseHandle(pi.hThread);
                    }
                    else {
                        // Если сам
CreateProcess не смог запуститься, выводим
код ошибки Windows
                        DWORD error =
GetLastError();
                        std::string errorMsg
= "Ошибка CreateProcess №: " +
std::to_string(error) +
                                    "\nПроверьте,
установлена ли в системе утилита hh.exe";
                        MessageBoxA(NULL,
errorMsg.c_str(), "Ошибка запуска", MB_OK |
MB_ICONERROR);
                    }
#endif
                }
            }
        }

        // Время показа экрана загрузки в
секундах (здесь 2.0 секунды)
        float elapsed =
loadingClock.getElapsedTime().asSeconds();
        float progress = elapsed / 2.0f;

        if (progress >= 1.f) {
            progress = 1.f;
            loading = false; // Завершаем
загрузку
        }

        // Обновляем ширину полосы в
зависимости от прогресса
        bar.setSize({ 400.f * progress, 16.f
});

        window.clear(sf::Color(24, 24, 26));
// Приятный темный фон
        window.draw(loadingText);
        window.draw(barBg);
        window.draw(bar);
        window.display();
    }

    auto backgroundTex =
std::make_unique<sf::Texture>();
    auto gameBgTex =
std::make_unique<sf::Texture>();
    auto buttonTex =
std::make_unique<sf::Texture>();
    auto allButtonTex =
std::make_unique<sf::Texture>();
    auto volumeOnTex =
std::make_unique<sf::Texture>();
    auto volumeOffTex =
std::make_unique<sf::Texture>();
    auto buttonFromAllTex =
std::make_unique<sf::Texture>();
    auto crystalTex =
std::make_unique<sf::Texture>();
    if (!backgroundTex-
>loadFromFile(BACKGROUND_PATH)) { std::cerr
<< "Background error\n"; return -1; }
    if (!gameBgTex-
>loadFromFile(GAME_BG_PATH)) { std::cerr <<
"Game background error\n"; return -1; }
    if (!buttonTex-
>loadFromFile(BUTTON_PATH)) { std::cerr <<
"Button texture error\n"; return -1; }
    if (!allButtonTex-
>loadFromFile(ALL_BUTTON_PATH)) { std::cerr
<< "All button texture error\n"; return -1;
}
    if (!volumeOnTex-
>loadFromFile(VOLUME_ON_PATH)) { std::cerr
<< "VolumeOn texture error\n"; return -1; }
    if (!volumeOffTex-
>loadFromFile(VOLUME_OFF_PATH)) { std::cerr
<< "VolumeOff texture error\n"; return -1; }
    if (!buttonFromAllTex-
>loadFromFile(BUTTON_FROM_ALL_PATH)) {
std::cerr << "ButtonFromAll texture
error\n"; return -1; }
    if (!crystalTex-
>loadFromFile(CRYSTAL_LINES_PATH)) {
std::cerr << "Crystal texture error\n";
return -1; }
```

```
    // ---- Размеры кристалла ----
    sf::Vector2u texSize = crystalTex-
>getSize();
    const float REFERENCE_WIDTH = 1280.f;
    const float REFERENCE_HEIGHT = 720.f;
    float scaleX = REFERENCE_WIDTH /
texSize.x;
    float scaleY = REFERENCE_HEIGHT /
texSize.y;

    // ---- Спрайты фонов ----
    auto background =
std::make_unique<sf::Sprite>(*backgroundTex)
;
    auto gameBackground =
std::make_unique<sf::Sprite>(*gameBgTex);
    background->setScale({
static_cast<float>(WINDOW_WIDTH) /
backgroundTex->getSize().x,

static_cast<float>(WINDOW_HEIGHT) /
backgroundTex->getSize().y });
    gameBackground->setScale({
static_cast<float>(WINDOW_WIDTH) /
gameBgTex->getSize().x,

static_cast<float>(WINDOW_HEIGHT) /
gameBgTex->getSize().y });

    // ---- Кристалл ----
    float crystalScale = 0.5f;
    auto crystalContour =
std::make_unique<sf::Sprite>(*crystalTex);
    crystalContour->setScale({ crystalScale,
crystalScale });
    float scaledWidth = texSize.x *
crystalScale;
    float scaledHeight = texSize.y *
crystalScale;
    sf::Vector2f crystalPos((WINDOW_WIDTH -
scaledWidth) / 2.f, (WINDOW_HEIGHT -
scaledHeight) / 2.f);
    crystalContour->setPosition(crystalPos +
GLOBAL_OFFSET);

    // ---- Фрагменты ----
    float fragmentScale = crystalScale *
1.02f;

std::vector<std::unique_ptr<sf::Texture>>
fragmentTextures(10);
    for (int i = 0; i < 10; ++i) {
        fragmentTextures[i] =
std::make_unique<sf::Texture>();
        std::string path =
CRYSTAL_FRAGMENT_PREFIX + std::to_string(i +
1) + ".png";
        if (!fragmentTextures[i]-
>loadFromFile(path))
            std::cerr << "Warning: can't
load " << path << std::endl;
    }
    sf::Vector2f fragmentPositions[10];
    for (int i = 0; i < 10; ++i) {
        float posX =
fragmentPositionsBase[i].x * scaleX;
        float posY =
fragmentPositionsBase[i].y * scaleY;
        fragmentPositions[i] = { posX, posY
};
    }
    std::vector<std::unique_ptr<sf::Sprite>>
fragmentSprites;
    fragmentSprites.reserve(10);
    for (int i = 0; i < 10; ++i) {
        auto spr =
std::make_unique<sf::Sprite>(*fragmentTextur
es[i]);
        spr->setScale({ fragmentScale,
fragmentScale });
        centerOrigin(*spr);
        sf::Vector2f pos =
fragmentPositions[i] * crystalScale +
crystalPos;
        spr->setPosition(pos + GLOBAL_OFFSET
+ fragmentOffsets[i]);

fragmentSprites.push_back(std::move(spr));
    }

    // ---- Звуки и музыка ----
    auto music =
std::make_unique<sf::Music>();
    int currentMusic = 1;
    std::string musicPaths[3] = {
MUSIC_PATH1, MUSIC_PATH2, MUSIC_PATH3 };
    if (!music->openFromFile(musicPaths[0]))
{ std::cerr << "Music error\n"; return -1; }
    music->setLooping(true);
    music->setVolume(100.f);
    music->play();
    bool musicOn = true;
    float volumeLevel = 100.f;

    auto clickBuffer =
std::make_unique<sf::SoundBuffer>();
    auto goodBuffer =
std::make_unique<sf::SoundBuffer>();
    auto badBuffer =
std::make_unique<sf::SoundBuffer>();
    auto resultBuffer =
std::make_unique<sf::SoundBuffer>();
    auto textSoundBuffer =
std::make_unique<sf::SoundBuffer>();
    auto resultEndBuffer =
std::make_unique<sf::SoundBuffer>();
    bool clickLoaded = clickBuffer-
>loadFromFile(CLICK_SOUND_PATH);
    bool goodLoaded = goodBuffer-
>loadFromFile(GOOD_SOUND_PATH);
    bool badLoaded = badBuffer-
>loadFromFile(BAD_SOUND_PATH);
    bool resultLoaded = resultBuffer-
>loadFromFile(RESULT_END_SOUND_PATH);
    bool textSoundLoaded = textSoundBuffer-
>loadFromFile(TEXT_SOUND_PATH);
    bool resultEndLoaded = resultEndBuffer-
>loadFromFile(RESULT_SOUND_PATH);

    auto clickSound =
std::make_unique<sf::Sound>(*clickBuffer);
    auto goodSound =
std::make_unique<sf::Sound>(*goodBuffer);
    auto badSound =
std::make_unique<sf::Sound>(*badBuffer);
    auto resultSound =
std::make_unique<sf::Sound>(*resultBuffer);
    auto textSound =
std::make_unique<sf::Sound>(*textSoundBuffer
);
    auto resultEndSound =
std::make_unique<sf::Sound>(*resultEndBuffer
);

    // ---- Функция создания кнопок ----
    auto makeButton = [&](const sf::Texture&
tex, float scale, sf::Vector2f pos) {
        auto btn =
std::make_unique<sf::Sprite>(tex);
        btn->setScale({ scale, scale });
        centerOrigin(*btn);
        btn->setPosition(pos);
        return btn;
        };
    std::unordered_map<sf::Sprite*, float>
originalScales;

    // ---- Создание всех кнопок и текстов -
---
```

```
    auto textBg =
std::make_unique<sf::RectangleShape>(sf::Vec
tor2f{ 900.f, 120.f });
    textBg->setFillColor(sf::Color(128, 0,
0, 80));
    textBg->setPosition({ 50.f,
WINDOW_HEIGHT - 150.f });

    auto animatedText =
std::make_unique<AnimatedText>(*font);
    animatedText->setCharacterSize(20);
    animatedText-
>setFillColor(sf::Color::White);
    animatedText->setString("Целостность
ядра нарушена.\nЧтобы восстановить кристалл,
необходимо извлечь 10 осколков из потока
данных.\nКаждый верный ответ — фрагмент
вашей коллекции.");
    animatedText->setPosition({ textBg-
>getPosition().x + 10, textBg-
>getPosition().y + 15 });
    animatedText-
>setTypingSound(textSound.get());

    float menuBtnScale = 360.f / buttonTex-
>getSize().x;
    auto menuStartBtn =
makeButton(*buttonTex, menuBtnScale, {
WINDOW_WIDTH / 2.f, 288.f });
    auto menuStartText =
std::make_unique<sf::Text>(*font, L"Начать",
32);
    menuStartText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*menuStartText,
*menuStartBtn);
    auto menuExitBtn =
makeButton(*buttonTex, menuBtnScale, {
WINDOW_WIDTH / 2.f, 418.f });
    auto menuExitText =
std::make_unique<sf::Text>(*font, L"Выход",
32);
    menuExitText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*menuExitText,
*menuExitBtn);
    auto menuSettingsBtn =
makeButton(*buttonTex, menuBtnScale, {
WINDOW_WIDTH / 2.f, 548.f });
    auto menuSettingsText =
std::make_unique<sf::Text>(*font,
L"Настройки", 28);
    menuSettingsText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*menuSettingsText,
*menuSettingsBtn);

    float startButtonScale = 360.f /
buttonTex->getSize().x;
    auto startButton =
makeButton(*buttonTex, startButtonScale,
        { WINDOW_WIDTH - 320.f + buttonTex-
>getSize().x * startButtonScale / 2.f,
         WINDOW_HEIGHT - 180.f + buttonTex-
>getSize().y * startButtonScale / 2.f });
    auto startButtonText =
std::make_unique<sf::Text>(*font, L"Старт",
30);
    startButtonText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*startButtonText,
*startButton);

    float topRightScale = 0.2f;
    auto topSettingsBtn =
makeButton(*allButtonTex, topRightScale, {
WINDOW_WIDTH - 60.f, 50.f });

    float crystalBtnScale = 0.125f;
    auto crystalViewBtn =
makeButton(*buttonFromAllTex,
crystalBtnScale,
        { WINDOW_WIDTH - 60.f, 50.f + 72.f +
10.f });
    auto crystalViewText =
std::make_unique<sf::Text>(*font,
L"Кристалл", 14);
    crystalViewText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*crystalViewText,
*crystalViewBtn);

    float checkScale = 250.f / buttonTex-
>getSize().x;
    auto checkButton =
makeButton(*buttonTex, checkScale,
        { WINDOW_WIDTH - 200.f + buttonTex-
>getSize().x * checkScale / 2.f,
         WINDOW_HEIGHT - 180.f + buttonTex-
>getSize().y * checkScale / 2.f });
    auto checkText =
std::make_unique<sf::Text>(*font,
L"Проверить", 20);
    checkText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*checkText,
*checkButton);

    auto nextButton = makeButton(*buttonTex,
checkScale,
        { WINDOW_WIDTH - 200.f + buttonTex-
>getSize().x * checkScale / 2.f,
         WINDOW_HEIGHT - 100.f + buttonTex-
>getSize().y * checkScale / 2.f });
    auto nextText =
std::make_unique<sf::Text>(*font, L"Далее",
20);
    nextText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*nextText,
*nextButton);

    // ---- Кнопка "Подсказка" ----
    float hintScale = 250.f / buttonTex-
>getSize().x;
    auto hintButton = makeButton(*buttonTex,
hintScale,
        { WINDOW_WIDTH - 200.f + buttonTex-
>getSize().x * hintScale / 2.f,
          WINDOW_HEIGHT - 180.f - buttonTex-
>getSize().y * hintScale / 2.f - 5.f });
    auto hintText =
std::make_unique<sf::Text>(*font,
L"Подсказка (3)", 22);
    hintText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*hintText,
*hintButton);

    auto backFromCrystalBtn =
makeButton(*buttonTex, 0.2f, { 100.f, 50.f
});
    auto backFromCrystalText =
std::make_unique<sf::Text>(*font, L"Назад",
24);
    backFromCrystalText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*backFromCrystalText,
*backFromCrystalBtn);

    auto backToMenuBtn =
makeButton(*buttonTex, 0.2f, { 100.f,
WINDOW_HEIGHT - 60.f });
    auto backToMenuText =
std::make_unique<sf::Text>(*font, L"В меню",
20);
    backToMenuText-
>setFillColor(sf::Color::White);
```

```
    centerTextOnButton(*backToMenuText,
*backToMenuBtn);

    // ---- НОВЫЕ НАСТРОЙКИ с ползунком
громкости ----
    // Ползунок громкости (слева)
    float sliderX = 250.f;
    float sliderY = 330.f;
    float sliderWidth = 300.f;
    float sliderHeight = 12.f;
    auto sliderBg =
std::make_unique<sf::RectangleShape>(sf::Vec
tor2f{ sliderWidth, sliderHeight });
    sliderBg->setFillColor(sf::Color(80, 80,
80));
    sliderBg->setOutlineThickness(1.f);
    sliderBg-
>setOutlineColor(sf::Color::White);
    sliderBg->setPosition({ sliderX, sliderY
});

    auto sliderFill =
std::make_unique<sf::RectangleShape>(sf::Vec
tor2f{ sliderWidth, sliderHeight });
    sliderFill->setFillColor(sf::Color(100,
200, 255));
    sliderFill->setPosition({ sliderX,
sliderY });

    auto sliderHandle =
std::make_unique<sf::CircleShape>(15.f);
    sliderHandle-
>setFillColor(sf::Color(200, 200, 200));
    sliderHandle->setOutlineThickness(2.f);
    sliderHandle-
>setOutlineColor(sf::Color::White);
    sliderHandle->setPosition({ sliderX +
volumeLevel / 100.f * sliderWidth - 15.f,
sliderY - 15.f + sliderHeight / 2.f });

    // Текст "Громкость"
    auto volumeLabel =
std::make_unique<sf::Text>(*font,
L"Громкость", 24);
    volumeLabel-
>setFillColor(sf::Color::White);
    volumeLabel->setPosition({ sliderX,
sliderY - 50 });

    // Блок выбора музыки - располагаем
справа, кнопки разнесены дальше
    float musicBtnScale = 0.15f;
    float musicY = 350.f;
    float musicSpacing = 200.f;
    float musicStartX = 700.f;
    auto musicBtn1 = makeButton(*buttonTex,
musicBtnScale, { musicStartX, musicY });
    auto musicBtn1Text =
std::make_unique<sf::Text>(*font, L"1", 24);
    musicBtn1Text-
>setFillColor(sf::Color::White);
    centerTextOnButton(*musicBtn1Text,
*musicBtn1);
    auto musicBtn2 = makeButton(*buttonTex,
musicBtnScale, { musicStartX + musicSpacing,
musicY });
    auto musicBtn2Text =
std::make_unique<sf::Text>(*font, L"2", 24);
    musicBtn2Text-
>setFillColor(sf::Color::White);
    centerTextOnButton(*musicBtn2Text,
*musicBtn2);
    auto musicBtn3 = makeButton(*buttonTex,
musicBtnScale, { musicStartX + 2 *
musicSpacing, musicY });
    auto musicBtn3Text =
std::make_unique<sf::Text>(*font, L"3", 24);
    musicBtn3Text-
>setFillColor(sf::Color::White);
    centerTextOnButton(*musicBtn3Text,
*musicBtn3);

    // Надпись "Выбрать музыку" - поднята
выше (было musicY-50, стало musicY-90)
    auto musicLabel =
std::make_unique<sf::Text>(*font, L"Выбрать
музыку", 22);
    musicLabel->setFillColor(sf::Color(200,
200, 200));
    musicLabel->setPosition({ musicStartX +
musicSpacing - 50.f, musicY - 90 });

    // Кнопки "Об авторе" и "Назад"
(уменьшены)
    float aboutScale = 0.2f;
    float backScale = 0.18f;
    auto aboutBtn = makeButton(*buttonTex,
aboutScale, { WINDOW_WIDTH / 2.f, 520.f });
    auto aboutText =
std::make_unique<sf::Text>(*font, L"Об
авторе", 20);
    aboutText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*aboutText,
*aboutBtn);
    auto settingsBackBtn =
makeButton(*buttonTex, backScale, {
WINDOW_WIDTH / 2.f, 610.f });
    auto settingsBackText =
std::make_unique<sf::Text>(*font, L"Назад",
20);
    settingsBackText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*settingsBackText,
*settingsBackBtn);

    // ---- Кнопки для финала ----
    auto finalStartBtn =
makeButton(*buttonTex, menuBtnScale, {
WINDOW_WIDTH - 200.f, WINDOW_HEIGHT - 80.f
});
    auto finalStartText =
std::make_unique<sf::Text>(*font, L"В меню",
32);
    finalStartText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*finalStartText,
*finalStartBtn);

    auto saveBtn = makeButton(*buttonTex,
menuBtnScale * 0.8f, { 200.f, WINDOW_HEIGHT
- 80.f });
    auto saveBtnText =
std::make_unique<sf::Text>(*font,
L"Сохранить", 28);
    saveBtnText-
>setFillColor(sf::Color::White);
    centerTextOnButton(*saveBtnText,
*saveBtn);

    // ---- Для подсказки (прямоугольник и
текст) ----
    auto hintOverlayBg =
std::make_unique<sf::RectangleShape>(sf::Vec
tor2f{ 800.f, 180.f });
    hintOverlayBg->setFillColor(sf::Color(0,
0, 0, 200));
    hintOverlayBg->setOutlineThickness(2.f);
    hintOverlayBg-
>setOutlineColor(sf::Color::White);
    hintOverlayBg->setPosition({
(WINDOW_WIDTH - 800.f) / 2.f, 300.f });

    auto hintOverlayText =
std::make_unique<sf::Text>(*font);
    hintOverlayText->setCharacterSize(18);
    hintOverlayText-
>setFillColor(sf::Color::Yellow);
```

```
    hintOverlayText->setPosition({
hintOverlayBg->getPosition().x + 20,
hintOverlayBg->getPosition().y + 20 });
    bool hintVisible = false;

    // Сохраняем масштабы для hover
(обновлены)
    originalScales[menuStartBtn.get()] =
menuBtnScale;
    originalScales[menuExitBtn.get()] =
menuBtnScale;
    originalScales[menuSettingsBtn.get()] =
menuBtnScale;
    originalScales[startButton.get()] =
startButtonScale;
    originalScales[topSettingsBtn.get()] =
topRightScale;
    originalScales[crystalViewBtn.get()] =
crystalBtnScale;
    originalScales[checkButton.get()] =
checkScale;
    originalScales[nextButton.get()] =
checkScale;
    originalScales[hintButton.get()] =
hintScale;
    originalScales[backFromCrystalBtn.get()]
= 0.2f;
    originalScales[backToMenuBtn.get()] =
0.2f;
    originalScales[aboutBtn.get()] =
aboutScale;
    originalScales[settingsBackBtn.get()] =
backScale;
    originalScales[musicBtn1.get()] =
musicBtnScale;
    originalScales[musicBtn2.get()] =
musicBtnScale;
    originalScales[musicBtn3.get()] =
musicBtnScale;
    originalScales[finalStartBtn.get()] =
menuBtnScale;
    originalScales[saveBtn.get()] =
menuBtnScale * 0.8f;

    // ---- Переменные состояния ----
    enum GameState { MENU, PRE_INSTRUCTION,
QUIZ, FINAL, SETTINGS, CRYSTAL_VIEW };
    GameState currentState = MENU;
    GameState previousState = MENU;
    bool textAnimationStarted = false;
    bool viewingCrystal = false;
    bool answerChecked = false;
    bool waitingForAnswer = true;
    bool fragmentFlying = false;
    bool aboutAuthor = false;
    bool finalCrystalComplete = false;
    bool resultSoundPlayed = false;
    bool isLastQuestion = false;

    int hintsLeft = 3;

    initQuestionPool();
    std::vector<Question> currentQuestions;
    int currentQuestionIdx = 0, score = 0;
    std::vector<bool> fragmentsCollected(10,
false);
    std::string resultMessage;

    bool flashEffect = false, shakeEffect =
false, redTextEffect = false;
    sf::Clock flashClock, shakeClock,
redTextClock;
    const float flashDuration = 0.1f,
shakeDuration = 0.2f, redTextDuration =
0.5f;
    sf::View originalView, shakeView;
    sf::Color originalTextColor =
sf::Color::White;

    auto flyingFragment =
std::make_unique<sf::Sprite>(*fragmentTextur
es[0]);
    sf::Vector2f flyStartPos, flyTargetPos;
    float flyProgress = 0.f;
    sf::Clock flyClock;
    const float flyDurationAnim = 0.4f;

    auto questionDisplay =
std::make_unique<sf::Text>(*font);
    auto taskNumber =
std::make_unique<sf::Text>(*font);
    questionDisplay->setCharacterSize(22);
    questionDisplay-
>setFillColor(sf::Color::White);
    questionDisplay->setPosition({ 100.f,
100.f });
    taskNumber->setCharacterSize(28);
    taskNumber->setFillColor(sf::Color(0,
180, 180, 200));
    taskNumber->setPosition({ 100.f, 60.f
});

    struct OptionPanel { sf::RectangleShape
rect; sf::Text text; int index; float
height; };
    std::vector<OptionPanel> optionPanels;
    int selectedOption = -1;

    auto inputBox =
std::make_unique<sf::RectangleShape>(sf::Vec
tor2f{ 400.f, 40.f });
    inputBox-
>setFillColor(sf::Color::Black);
    inputBox->setOutlineThickness(2.f);
    inputBox-
>setOutlineColor(sf::Color::White);
    auto inputText =
std::make_unique<sf::Text>(*font);
    inputText->setCharacterSize(20);
    inputText-
>setFillColor(sf::Color::White);
    std::string userFill;
    bool inputActive = false;

    auto codeBackground =
std::make_unique<sf::RectangleShape>();
    codeBackground-
>setFillColor(sf::Color(30, 30, 30, 220));
    codeBackground-
>setOutlineThickness(1.f);
    codeBackground-
>setOutlineColor(sf::Color::White);
    std::vector<sf::Text> codeLineTexts;
    std::vector<std::string>
currentCodeLines;
    int editingLine = -1;
    bool editing = false;
    std::string editedLineContent;
    int cursorPos = 0;
    sf::Clock cursorBlinkClock;
    bool cursorVisible = true;

    struct OrderItem { sf::RectangleShape
rect; sf::Text text; int correctOrder; int
currentPos; bool dragging; sf::Vector2f
dragOffset; };
    std::vector<OrderItem> orderItems;
    int draggingIndex = -1;

    auto correctAnswerText =
std::make_unique<sf::Text>(*font);
    correctAnswerText->setCharacterSize(18);
    correctAnswerText-
>setFillColor(sf::Color::Black);

    auto fadeRect =
std::make_unique<sf::RectangleShape>(sf::Vec
tor2f{ static_cast<float>(WINDOW_WIDTH),
static_cast<float>(WINDOW_HEIGHT) });
```

```
    fadeRect->setFillColor(sf::Color(0, 0,
0, 0));
    bool fading = false;
    bool fadeIn = false;
    sf::Clock fadeClock;
    const float fadeDurationSec = 0.5f;
    GameState nextStateAfterFade = MENU;

    // ---- Лямбда для обновления текста
подсказки ----
    auto updateHintButtonText = [&]() {
        if (hintsLeft > 0) {
            hintText-
>setString(utf8_to_sfstring("Подсказка (" +
std::to_string(hintsLeft) + ")"));
        }
        else {
            hintText->setString(L"Подсказок
нет");
        }
        centerTextOnButton(*hintText,
*hintButton);
        };

    // ---- Лямбда для обновления ползунка -
---
    auto updateSlider = [&]() {
        sliderFill->setSize({ volumeLevel /
100.f * sliderWidth, sliderHeight });
        sliderHandle->setPosition({ sliderX
+ volumeLevel / 100.f * sliderWidth - 15.f,
sliderY - 15.f + sliderHeight / 2.f });
        music->setVolume(volumeLevel);
        musicOn = (volumeLevel > 0.f);
        };

    // ---- Лямбда вычисления позиции
курсора по клику ----
    auto getCursorPosFromClick = [&](const
sf::Text& text, const sf::Vector2f&
clickPos) -> int {
        std::string str =
text.getString().toAnsiString();
        float x = clickPos.x -
text.getPosition().x;
        for (size_t i = 0; i <=
str.length(); ++i) {
            std::string sub = str.substr(0,
i);
            sf::Text subText(*font,
utf8_to_sfstring(sub),
text.getCharacterSize());
            float w =
subText.getLocalBounds().size.x;
            if (x < w) return
static_cast<int>(i);
        }
        return
static_cast<int>(str.length());
        };

    // ---- Лямбда загрузки вопроса ----
    auto loadQuestion = [&]() {
        if (currentQuestionIdx >=
static_cast<int>(currentQuestions.size())) {
            currentState = FINAL;
            finalCrystalComplete = false;
            resultSoundPlayed = false;
            return;
        }
        const Question& q =
currentQuestions[currentQuestionIdx];
        std::string wrappedQ =
wrapText(q.questionText, *font, 22,
WINDOW_WIDTH - 200);
        questionDisplay-
>setString(utf8_to_sfstring(wrappedQ));
        taskNumber-
>setString(utf8_to_sfstring("Задание " +
std::to_string(currentQuestionIdx + 1) + "
из 10"));
        optionPanels.clear(); selectedOption
= -1;
        codeLineTexts.clear();
currentCodeLines.clear();
        orderItems.clear();
userFill.clear(); inputActive = false;
editing = false; editingLine = -1;
draggingIndex = -1;
        cursorPos = 0; cursorVisible = true;
cursorBlinkClock.restart();
        correctAnswerText->setString("");
        answerChecked = false;
        waitingForAnswer = true;
        resultMessage.clear();
        hintVisible = false;
        if (redTextEffect) questionDisplay-
>setFillColor(originalTextColor);
        redTextEffect = false;
        isLastQuestion = (currentQuestionIdx
== 9);
        if (isLastQuestion) {
            nextText-
>setString(L"Завершить");
        }
        else {
            nextText->setString(L"Далее");
        }
        updateHintButtonText();

        if (q.type == QuestionType::CHOICE)
{
            float startX = (WINDOW_WIDTH -
PANEL_WIDTH) / 2.f;
            float startY = questionDisplay-
>getPosition().y + 60 + questionDisplay-
>getLocalBounds().size.y + 20;
            float stepY = 0;
            for (size_t i = 0; i <
q.options.size(); ++i) {
                std::string wrappedOpt =
wrapText(q.options[i], *font, 18,
PANEL_WIDTH - 20);
                sf::Text txt(*font,
utf8_to_sfstring(wrappedOpt), 18);

txt.setFillColor(TEXT_COLOR);
                sf::FloatRect tb =
txt.getLocalBounds();
                int lines = 1;
                for (char c : wrappedOpt) if
(c == '\n') lines++;
                float lineHeight = 22.f;
                float panelHeight = lines *
lineHeight + 10.f;
                sf::RectangleShape rect({
PANEL_WIDTH, panelHeight });

rect.setFillColor(PANEL_COLOR);
                rect.setPosition({ startX,
startY + stepY });
                txt.setPosition({
rect.getPosition().x + 10,

rect.getPosition().y + (panelHeight -
tb.size.y) / 2.f - tb.position.y });
                optionPanels.push_back({
rect, txt, static_cast<int>(i), panelHeight
});
                stepY += panelHeight + 10.f;
            }
        }
        else if (q.type ==
QuestionType::FILL_CODE) {
            std::istringstream
iss(q.codeSnippet);
            std::string line;
```

```
            float y = questionDisplay-
>getPosition().y + 60 + questionDisplay-
>getLocalBounds().size.y + 20;
            int idx = 0;
            while (std::getline(iss, line))
{
                sf::Text txt(*font,
utf8_to_sfstring(line), 16);

txt.setFillColor(sf::Color::White);
                txt.setPosition({ 105.f, y +
static_cast<float>(idx) * 30 + 2 });

codeLineTexts.push_back(txt);
                ++idx;
            }
            float totalHeight =
static_cast<float>(idx) * 30;
            codeBackground->setSize({
static_cast<float>(WINDOW_WIDTH) - 200,
totalHeight });
            codeBackground->setPosition({
100.f, y });
            inputBox->setSize({ 400.f, 40.f
});
            inputBox->setPosition({
(WINDOW_WIDTH - 400) / 2.f, y + totalHeight
+ 50 });
            inputText->setPosition({
inputBox->getPosition().x + 10, inputBox-
>getPosition().y + 5 });
            inputText->setString("");
            userFill = "";
        }
        else if (q.type ==
QuestionType::FIND_ERROR) {
            std::istringstream
iss(q.brokenCode);
            std::string line;
            float y = questionDisplay-
>getPosition().y + 60 + questionDisplay-
>getLocalBounds().size.y + 20;
            int idx = 0;
            while (std::getline(iss, line))
{
                sf::Text txt(*font,
utf8_to_sfstring(line), 16);

txt.setFillColor(sf::Color::White);
                txt.setPosition({ 105.f, y +
static_cast<float>(idx) * 30 + 2 });

codeLineTexts.push_back(txt);

currentCodeLines.push_back(line);
                ++idx;
            }
            float totalHeight =
static_cast<float>(idx) * 30;
            codeBackground->setSize({
static_cast<float>(WINDOW_WIDTH) - 200,
totalHeight });
            codeBackground->setPosition({
100.f, y });
            editingLine = -1;
            editing = false;
            cursorPos = 0;
            cursorVisible = true;
            cursorBlinkClock.restart();
        }
        else if (q.type ==
QuestionType::ORDER) {
            std::vector<OrderLine>
shuffledLines = q.lines;
            auto seed = static_cast<unsigned
int>(std::chrono::steady_clock::now().time_s
ince_epoch().count());
            std::default_random_engine
rng(seed);
std::shuffle(shuffledLines.begin(),
shuffledLines.end(), rng);
            float y = questionDisplay-
>getPosition().y + 60 + questionDisplay-
>getLocalBounds().size.y + 20;
            float x = (WINDOW_WIDTH - 600) /
2.f;
            for (size_t i = 0; i <
shuffledLines.size(); ++i) {
                sf::RectangleShape rect({
600.f, 40.f });

rect.setFillColor(PANEL_COLOR);
                rect.setPosition({ x, y +
static_cast<float>(i) * 50 });
                sf::Text txt(*font,
utf8_to_sfstring(shuffledLines[i].text),
18);

txt.setFillColor(TEXT_COLOR);
                sf::FloatRect tb =
txt.getLocalBounds();
                txt.setPosition({
rect.getPosition().x + 10,

rect.getPosition().y + (40 - tb.size.y) /
2.f - tb.position.y });
                orderItems.push_back({ rect,
txt, shuffledLines[i].correctOrder,
static_cast<int>(i), false, {0,0} });
            }
        }
        };

    // ---- Лямбда проверки ответа ----
    auto checkAnswer = [&]() {
        if (!waitingForAnswer) return;
        const Question& q =
currentQuestions[currentQuestionIdx];
        bool correct = false;
        for (auto& t : codeLineTexts)
t.setFillColor(sf::Color::White);
        if (q.type == QuestionType::CHOICE)
{
            bool isCorrect = false;
            for (int correctOpt :
q.correctOptions) {
                if (selectedOption ==
correctOpt) { isCorrect = true; break; }
            }
            correct = isCorrect;
            for (auto& p : optionPanels) {
                if (p.index ==
selectedOption) p.text.setFillColor(correct
? TEXT_CORRECT_COLOR : TEXT_WRONG_COLOR);
                else {
                    bool isCorrectOpt =
false;
                    for (int c :
q.correctOptions) if (c == p.index) {
isCorrectOpt = true; break; }
                    if (isCorrectOpt &&
!correct)
p.text.setFillColor(TEXT_CORRECT_COLOR);
                    else
p.text.setFillColor(TEXT_COLOR);
                }
            }
            if (!correct) {
                std::string correctStr =
"Правильный ответ: ";
                for (size_t i = 0; i <
q.correctOptions.size(); ++i) {
                    if (i > 0) correctStr +=
", ";
                    correctStr +=
q.options[q.correctOptions[i]];
                }
```

```
                std::string wrapped =
wrapText(correctStr, *font, 18, WINDOW_WIDTH
- 100);
                correctAnswerText-
>setString(utf8_to_sfstring(wrapped));
                auto bounds =
correctAnswerText->getGlobalBounds();
                correctAnswerText-
>setPosition({ (WINDOW_WIDTH -
bounds.size.x) / 2.f, WINDOW_HEIGHT - 200.f
});
            }
        }
        else if (q.type ==
QuestionType::FILL_CODE) {
            std::string userNorm =
normalizeString(userFill);
            std::string expectedNorm =
normalizeString(q.expectedFill);
            if (userNorm == expectedNorm)
correct = true;
            inputText->setFillColor(correct
? sf::Color::Green : sf::Color::Red);
            if (!correct) {
                std::string wrapped =
wrapText("Правильный ответ: " +
q.expectedFill, *font, 18, WINDOW_WIDTH -
100);
                correctAnswerText-
>setString(utf8_to_sfstring(wrapped));
                auto bounds =
correctAnswerText->getGlobalBounds();
                correctAnswerText-
>setPosition({ (WINDOW_WIDTH -
bounds.size.x) / 2.f, WINDOW_HEIGHT - 200.f
});
            }
        }
        else if (q.type ==
QuestionType::FIND_ERROR) {
            std::string userCode;
            for (const auto& line :
currentCodeLines) userCode += line + "\n";
            if (!userCode.empty() &&
userCode.back() == '\n')
userCode.pop_back();
            if (normalizeCode(userCode) ==
normalizeCode(q.fixedCode)) correct = true;
            if (correct) for (auto& t :
codeLineTexts)
t.setFillColor(sf::Color::Green);
            else {
                for (auto& t :
codeLineTexts)
t.setFillColor(sf::Color::Red);
                std::string wrapped =
wrapText("Правильный код:\n" + q.fixedCode,
*font, 18, WINDOW_WIDTH - 100);
                correctAnswerText-
>setString(utf8_to_sfstring(wrapped));
                auto bounds =
correctAnswerText->getGlobalBounds();
                correctAnswerText-
>setPosition({ (WINDOW_WIDTH -
bounds.size.x) / 2.f, WINDOW_HEIGHT - 200.f
});
            }
        }
        else if (q.type ==
QuestionType::ORDER) {
            bool ok = true;
            for (size_t i = 0; i <
orderItems.size(); ++i) if
(orderItems[i].correctOrder !=
static_cast<int>(i)) { ok = false; break; }
            correct = ok;
            if (!correct) {
                std::string correctOrderStr
= "Правильный порядок:\n";
                std::vector<std::pair<int,
std::string>> sorted;
                for (const auto& item :
orderItems) {

sorted.emplace_back(item.correctOrder,
item.text.getString().toAnsiString());
                }
                std::sort(sorted.begin(),
sorted.end());
                for (size_t i = 0; i <
sorted.size(); ++i) correctOrderStr +=
std::to_string(i + 1) + ". " +
sorted[i].second + "\n";
                std::string wrapped =
wrapText(correctOrderStr, *font, 18,
WINDOW_WIDTH - 100);
                correctAnswerText-
>setString(utf8_to_sfstring(wrapped));
                auto bounds =
correctAnswerText->getGlobalBounds();
                correctAnswerText-
>setPosition({ (WINDOW_WIDTH -
bounds.size.x) / 2.f, WINDOW_HEIGHT - 200.f
});
                for (auto& item :
orderItems)
item.text.setFillColor(sf::Color::Red);
            }
            else for (auto& item :
orderItems)
item.text.setFillColor(sf::Color::Green);
        }

        if (correct) {
            score++;

fragmentsCollected[currentQuestionIdx] =
true;
            resultMessage = "Правильно!";
            if (goodLoaded) goodSound-
>play();
            flashEffect = true;
            flashClock.restart();
            if (currentQuestionIdx < 10) {
                flyingFragment-
>setTexture(*fragmentTextures[currentQuestio
nIdx]);
                flyingFragment->setScale({
fragmentScale, fragmentScale });

centerOrigin(*flyingFragment);
                flyStartPos = { WINDOW_WIDTH
/ 2.f, WINDOW_HEIGHT / 2.f };
                flyTargetPos =
fragmentSprites[currentQuestionIdx]-
>getPosition();
                flyProgress = 0.f;
                fragmentFlying = true;
                flyClock.restart();
            }
        }
        else {
            resultMessage = "Неправильно!";
            if (badLoaded) badSound->play();
            shakeEffect = true;
            originalView = window.getView();
            shakeView = originalView;
            shakeClock.restart();
            redTextEffect = true;
            redTextClock.restart();
            originalTextColor =
questionDisplay->getFillColor();
            questionDisplay-
>setFillColor(sf::Color::Red);
        }
        waitingForAnswer = false;
        answerChecked = true;
        };
```

```
    // ---- Лямбда завершения игры ----
    auto finishGame = [&]() {
        currentState = FINAL;
        finalCrystalComplete = false;
        resultSoundPlayed = false;
        };

    // ---- Лямбда перехода к следующему
вопросу ----
    auto proceedNext = [&]() {
        if (!answerChecked) return;
        if (isLastQuestion) {
            finishGame();
            return;
        }
        if (currentQuestionIdx + 1 >=
static_cast<int>(currentQuestions.size())) {
            finishGame();
            return;
        }
        currentQuestionIdx++;
        loadQuestion();
        answerChecked = false;
        waitingForAnswer = true;
        resultMessage.clear();
        if (redTextEffect) questionDisplay-
>setFillColor(originalTextColor);
        redTextEffect = false;
        };

    // ---- Лямбда старта игры ----
    auto startGame = [&]() {
        currentQuestions =
selectRandomQuestions();
        currentQuestionIdx = 0;
        score = 0;
        fragmentsCollected.assign(10,
false);
        hintsLeft = 3;
        loadQuestion();
        currentState = QUIZ;
        viewingCrystal = false;
        answerChecked = false;
        resultSoundPlayed = false;
        };

    // ---- Эффект затемнения ----
    auto startFadeTo = [&](GameState
newState) {
        fading = true;
        fadeIn = true;
        fadeClock.restart();
        nextStateAfterFade = newState;
        };

    // ---- Главный цикл ----
    bool draggingSlider = false;
    while (window.isOpen()) {

        sf::Vector2i mousePosInt =
sf::Mouse::getPosition(window);
        sf::Vector2f mousePos =
window.mapPixelToCoords(mousePosInt);

        // ---- Обработка событий ----
        while (auto event =
window.pollEvent()) {
            if (event-
>is<sf::Event::Closed>()) window.close();

            if (auto* mp = event-
>getIf<sf::Event::MouseButtonPressed>()) {
                if (mp->button ==
sf::Mouse::Button::Left) {
                    sf::Vector2f clickPos =
window.mapPixelToCoords({ mp->position.x,
mp->position.y });
                    bool clicked = false;

                    if (currentState == MENU
&& !fading) {
                        if
(isPointInSpriteWithPadding(*menuStartBtn,
clickPos)) {

startFadeTo(PRE_INSTRUCTION);
                            clicked = true;
                        }
                        else if
(isPointInSpriteWithPadding(*menuExitBtn,
clickPos)) {
                            window.close();
                            clicked = true;
                        }
                        else if
(isPointInSpriteWithPadding(*menuSettingsBtn
, clickPos)) {
                            previousState =
currentState;
                            currentState =
SETTINGS;
                            aboutAuthor =
false;
                            clicked = true;
                        }
                    }
                    else if (currentState ==
PRE_INSTRUCTION && !fading) {
                        if
(isPointInSpriteWithPadding(*startButton,
clickPos)) {
                            startGame();
                            clicked = true;
                        }
                        else if
(isPointInSpriteWithPadding(*topSettingsBtn,
clickPos)) {
                            previousState =
currentState;
                            currentState =
SETTINGS;
                            aboutAuthor =
false;
                            clicked = true;
                        }
                    }
                    else if (currentState ==
QUIZ && !fading) {
                        if (!viewingCrystal)
{
                            if
(isPointInSpriteWithPadding(*crystalViewBtn,
clickPos)) {

viewingCrystal = true;
                                clicked =
true;
                            }
                            else if
(isPointInSpriteWithPadding(*topSettingsBtn,
clickPos)) {

previousState = currentState;
                                currentState
= SETTINGS;
                                aboutAuthor
= false;
                                clicked =
true;
                            }
                            else if
(isPointInSpriteWithPadding(*backToMenuBtn,
clickPos)) {
                                currentState
= MENU;
                                music-
>play();
                                clicked =
true;
```

```
                            }
                            if
(!isLastQuestion || !answerChecked) {
                                if
(isPointInSpriteWithPadding(*checkButton,
clickPos) && !answerChecked &&
!fragmentFlying) {

checkAnswer();
                                    clicked
= true;
                                }
                            }
                            if
(isPointInSpriteWithPadding(*nextButton,
clickPos) && answerChecked &&
!fragmentFlying) {

proceedNext();
                                clicked =
true;
                            }
                            if
(!isLastQuestion || !answerChecked) {
                                if
(isPointInSpriteWithPadding(*hintButton,
clickPos) && !answerChecked &&
!fragmentFlying && hintsLeft > 0) {
                                    const
Question& q =
currentQuestions[currentQuestionIdx];
                                    if
(q.type == QuestionType::FIND_ERROR) {

hintVisible = !hintVisible;
                                        if
(hintVisible) {

std::string wrappedHint =
wrapText("Подсказка: " + q.hint, *font, 18,
760);

hintOverlayText-
>setString(utf8_to_sfstring(wrappedHint));

hintsLeft--;

updateHintButtonText();
                                        }

clicked = true;
                                    }
                                }
                            }
                            if (hintVisible
&& containsPoint(hintOverlayBg-
>getGlobalBounds(), clickPos)) {
                                hintVisible
= false;
                                clicked =
true;
                            }

                            const Question&
q = currentQuestions[currentQuestionIdx];
                            if
(!answerChecked) {
                                if (q.type
== QuestionType::CHOICE) {
                                    for
(auto& p : optionPanels) {
                                        if
(containsPoint(p.rect.getGlobalBounds(),
clickPos)) {

selectedOption = p.index;

for (auto& opt : optionPanels)
opt.text.setFillColor(TEXT_COLOR);

p.text.setFillColor(TEXT_SELECTED_COLOR);

clicked = true;

break;
                                        }
                                    }
                                }
                                else if
(q.type == QuestionType::FILL_CODE) {
                                    if
(containsPoint(inputBox->getGlobalBounds(),
clickPos)) inputActive = true;
                                    else
inputActive = false;
                                }
                                else if
(q.type == QuestionType::FIND_ERROR) {
                                    float
yStart = codeBackground->getPosition().y,
lineHeight = 30.f;
                                    for
(size_t i = 0; i < codeLineTexts.size();
++i) {

sf::FloatRect lineRect({ codeBackground-
>getPosition().x, yStart +
static_cast<float>(i) * lineHeight },

{ codeBackground->getSize().x, lineHeight
});
                                        if
(containsPoint(lineRect, clickPos)) {

editingLine = static_cast<int>(i);

editedLineContent = currentCodeLines[i];

cursorPos =
getCursorPosFromClick(codeLineTexts[i],
clickPos);

editing = true;

cursorVisible = true;

cursorBlinkClock.restart();

clicked = true;

break;
                                        }
                                    }
                                }
                                else if
(q.type == QuestionType::ORDER) {
                                    for
(size_t i = 0; i < orderItems.size(); ++i) {
                                        if
(containsPoint(orderItems[i].rect.getGlobalB
ounds(), clickPos)) {

draggingIndex = static_cast<int>(i);

orderItems[i].dragging = true;

orderItems[i].dragOffset = clickPos -
orderItems[i].rect.getPosition();

clicked = true;

break;
                                        }
                                    }
                                }
                            }
                        }
                        else { //
viewingCrystal
```

```
                            if
(isPointInSpriteWithPadding(*backFromCrystal
Btn, clickPos)) {

viewingCrystal = false;
                                clicked =
true;
                            }
                            else if
(isPointInSpriteWithPadding(*topSettingsBtn,
clickPos)) {

previousState = currentState;
                                currentState
= SETTINGS;
                                aboutAuthor
= false;
                                clicked =
true;
                            }
                        }
                    }
                    else if (currentState ==
SETTINGS && !fading) {
                        // Ползунок
                        sf::FloatRect
sliderRect(sliderBg->getPosition(),
sliderBg->getSize());
                        sf::FloatRect
handleRect(sliderHandle->getPosition(), {
30.f, 30.f });
                        if
(sliderRect.contains(clickPos) ||
handleRect.contains(clickPos)) {
                            draggingSlider =
true;
                            float newX =
std::max(sliderX, std::min(sliderX +
sliderWidth, clickPos.x));
                            volumeLevel =
(newX - sliderX) / sliderWidth * 100.f;
                            volumeLevel =
std::max(0.f, std::min(100.f, volumeLevel));
                            updateSlider();
                            clicked = true;
                        }
                        else if
(isPointInSpriteWithPadding(*aboutBtn,
clickPos)) {
                            aboutAuthor =
true;
                            clicked = true;
                        }
                        else if
(isPointInSpriteWithPadding(*settingsBackBtn
, clickPos)) {
                            if (aboutAuthor)
{
                                aboutAuthor
= false;
                            }
                            else {
                                currentState
= previousState;
                            }
                            clicked = true;
                        }
                        else if
(isPointInSpriteWithPadding(*musicBtn1,
clickPos)) {
                            currentMusic =
1;
                            if (musicOn) {
                                music-
>stop();
                                if (music-
>openFromFile(musicPaths[0])) music->play();
                                music-
>setVolume(volumeLevel);
                            }
                            clicked = true;
                        }
                        else if
(isPointInSpriteWithPadding(*musicBtn2,
clickPos)) {
                            currentMusic =
2;
                            if (musicOn) {
                                music-
>stop();
                                if (music-
>openFromFile(musicPaths[1])) music->play();
                                music-
>setVolume(volumeLevel);
                            }
                            clicked = true;
                        }
                        else if
(isPointInSpriteWithPadding(*musicBtn3,
clickPos)) {
                            currentMusic =
3;
                            if (musicOn) {
                                music-
>stop();
                                if (music-
>openFromFile(musicPaths[2])) music->play();
                                music-
>setVolume(volumeLevel);
                            }
                            clicked = true;
                        }
                    }
                    else if (currentState ==
FINAL && !fading) {
                        if
(isPointInSpriteWithPadding(*finalStartBtn,
clickPos)) {

startFadeTo(MENU);
                            clicked = true;
                        }
                        else if
(isPointInSpriteWithPadding(*saveBtn,
clickPos)) {
#ifdef _WIN32
                            OPENFILENAMEA
ofn = { 0 };
                            char
fileName[MAX_PATH] = "result.txt";
                            ofn.lStructSize
= sizeof(ofn);
                            ofn.hwndOwner =
reinterpret_cast<HWND>(window.getNativeHandl
e());
                            ofn.lpstrFilter
= "Text Files\0*.txt\0All Files\0*.*\0";
                            ofn.lpstrFile =
fileName;
                            ofn.nMaxFile =
MAX_PATH;
                            ofn.lpstrTitle =
"Сохранить результат";
                            ofn.Flags =
OFN_OVERWRITEPROMPT | OFN_PATHMUSTEXIST;
                            if
(GetSaveFileNameA(&ofn)) {
                                std::string
filename = fileName;

std::ofstream file(filename);
                                if
(file.is_open()) {
                                    auto now
= std::chrono::system_clock::now();

std::time_t now_c =
std::chrono::system_clock::to_time_t(now);
                                    file <<
"Результат игры \"Массивы в языке C++\"\n";
```

```
                                    file <<
"Дата: " << std::ctime(&now_c);
                                    file <<
"Очки: " << score << " из 10\n";

std::string msg;
                                    if
(score >= 9) msg = "Абсолютный резонанс.
Мастер.";
                                    else if
(score >= 7) msg = "Высокоэффективный
синтез.";
                                    else if
(score >= 4) msg = "Стабильная добыча.";
                                    else msg
= "Критический сбой энергосети.";
                                    file <<
"Итог: " << msg << std::endl;

file.close();

resultMessage = "Сохранено в " + filename;
                                }
                                else {

resultMessage = "Ошибка сохранения!";
                                }
                            }
#else
                            resultMessage =
"Сохранение доступно только в Windows";
#endif
                            clicked = true;
                        }
                    }
                    if (clicked &&
clickLoaded && currentState != SETTINGS)
clickSound->play();
                }
            }

            // ---- Обработка отпускания
кнопки мыши для ползунка ----
            if (auto* mr = event-
>getIf<sf::Event::MouseButtonReleased>()) {
                if (mr->button ==
sf::Mouse::Button::Left) {
                    draggingSlider = false;
                }
            }

            // ---- Движение мыши для
ползунка ----
            if (auto* mm = event-
>getIf<sf::Event::MouseMoved>()) {
                if (draggingSlider &&
currentState == SETTINGS) {
                    float newX =
std::max(sliderX, std::min(sliderX +
sliderWidth, static_cast<float>(mm-
>position.x)));
                    volumeLevel = (newX -
sliderX) / sliderWidth * 100.f;
                    volumeLevel =
std::max(0.f, std::min(100.f, volumeLevel));
                    updateSlider();
                }
            }

            // ---- Обработка клавиатуры ---
-
            if (currentState == QUIZ &&
!viewingCrystal && !fading) {
                if (auto* ke = event-
>getIf<sf::Event::KeyPressed>()) {
                    if (!answerChecked &&
currentQuestions[currentQuestionIdx].type ==
QuestionType::CHOICE) {
                        int num = -1;
                        if (ke->code ==
sf::Keyboard::Key::Num1 || ke->code ==
sf::Keyboard::Key::Numpad1) num = 0;
                        else if (ke->code ==
sf::Keyboard::Key::Num2 || ke->code ==
sf::Keyboard::Key::Numpad2) num = 1;
                        else if (ke->code ==
sf::Keyboard::Key::Num3 || ke->code ==
sf::Keyboard::Key::Numpad3) num = 2;
                        else if (ke->code ==
sf::Keyboard::Key::Num4 || ke->code ==
sf::Keyboard::Key::Numpad4) num = 3;
                        if (num >= 0 && num
< static_cast<int>(optionPanels.size())) {
                            selectedOption =
num;
                            for (auto& opt :
optionPanels)
opt.text.setFillColor(TEXT_COLOR);

optionPanels[num].text.setFillColor(TEXT_SEL
ECTED_COLOR);
                        }
                    }

                    if (!answerChecked &&
editing &&
currentQuestions[currentQuestionIdx].type ==
QuestionType::FIND_ERROR) {
                        if (ke->code ==
sf::Keyboard::Key::Left) {
                            if (cursorPos >
0) cursorPos--;
                        }
                        else if (ke->code ==
sf::Keyboard::Key::Right) {
                            if (cursorPos <
static_cast<int>(editedLineContent.length())
) cursorPos++;
                        }
                        else if (ke->code ==
sf::Keyboard::Key::Delete) {
                            if (cursorPos <
static_cast<int>(editedLineContent.length())
) {

editedLineContent.erase(cursorPos, 1);

codeLineTexts[editingLine].setString(utf8_to
_sfstring(editedLineContent));

codeLineTexts[editingLine].setFillColor(sf::
Color::Yellow);
                            }
                        }
                    }

                    if (ke->code ==
sf::Keyboard::Key::Enter) {
                        if (!answerChecked
&& !fragmentFlying) {
                            checkAnswer();
                        }
                        else if
(answerChecked && !fragmentFlying) {
                            proceedNext();
                        }
                    }
                    if (ke->code ==
sf::Keyboard::Key::Escape) {
                        if (currentState ==
QUIZ) {
                            if
(viewingCrystal) viewingCrystal = false;
                            else {
                                currentState
= MENU;
                                music-
>play();
                            }
```

```
                        }
                        else if
(currentState == SETTINGS) {
                            currentState =
previousState;
                            aboutAuthor =
false;
                        }
                        else if
(currentState == PRE_INSTRUCTION) {
                            currentState =
MENU;
                            music->play();
                        }
                        else if
(currentState == FINAL) {
                            // ничего не
делаем
                        }
                    }
                }

                // ---- Ввод для FILL_CODE -
---
                if (inputActive &&
!answerChecked &&
currentQuestions[currentQuestionIdx].type ==
QuestionType::FILL_CODE) {
                    if (auto* te = event-
>getIf<sf::Event::TextEntered>()) {
                        if (te->unicode ==
8) { if (!userFill.empty())
userFill.pop_back(); }
                        else if (te->unicode
>= 32 && te->unicode < 128) userFill +=
static_cast<char>(te->unicode);
                        inputText-
>setString(userFill);
                    }
                }

                // ---- Ввод для FIND_ERROR
----
                if (editing &&
!answerChecked &&
currentQuestions[currentQuestionIdx].type ==
QuestionType::FIND_ERROR) {
                    if (auto* te = event-
>getIf<sf::Event::TextEntered>()) {
                        if (te->unicode ==
8) {
                            if (cursorPos >
0) {

editedLineContent.erase(cursorPos - 1, 1);
                                cursorPos--;

codeLineTexts[editingLine].setString(utf8_to
_sfstring(editedLineContent));

codeLineTexts[editingLine].setFillColor(sf::
Color::Yellow);
                            }
                        }
                        else if (te->unicode
== 13) {

currentCodeLines[editingLine] =
editedLineContent;

codeLineTexts[editingLine].setString(utf8_to
_sfstring(editedLineContent));

codeLineTexts[editingLine].setFillColor(sf::
Color::Yellow);
                            editing = false;
                            editingLine = -
1;
                        }
                        else if (te->unicode
>= 32 && te->unicode < 128) {

editedLineContent.insert(cursorPos, 1,
static_cast<char>(te->unicode));
                            cursorPos++;

codeLineTexts[editingLine].setString(utf8_to
_sfstring(editedLineContent));

codeLineTexts[editingLine].setFillColor(sf::
Color::Yellow);
                        }
                    }
                }
            }
        }

        // ---- Обновление анимаций ----
        if (flashEffect &&
flashClock.getElapsedTime().asSeconds() >=
flashDuration) flashEffect = false;
        if (shakeEffect &&
shakeClock.getElapsedTime().asSeconds() >=
shakeDuration) {
            shakeEffect = false;
            window.setView(originalView);
        }
        else if (shakeEffect) {
            float intensity = 5.f;
            float offX = (std::rand() %
static_cast<int>(2 * intensity)) -
intensity;
            float offY = (std::rand() %
static_cast<int>(2 * intensity)) -
intensity;
            shakeView.setCenter({
originalView.getCenter().x + offX,
originalView.getCenter().y + offY });
            window.setView(shakeView);
        }
        if (redTextEffect &&
redTextClock.getElapsedTime().asSeconds() >=
redTextDuration) {
            redTextEffect = false;
            questionDisplay-
>setFillColor(originalTextColor);
        }

        // Мигание курсора
        if (editing &&
cursorBlinkClock.getElapsedTime().asSeconds(
) > 0.5f) {
            cursorVisible = !cursorVisible;
            cursorBlinkClock.restart();
        }

        // Drag-and-drop для ORDER
        if (draggingIndex != -1 &&
currentState == QUIZ && !viewingCrystal &&
!answerChecked) {

orderItems[draggingIndex].rect.setPosition(m
ousePos -
orderItems[draggingIndex].dragOffset);
            if
(!sf::Mouse::isButtonPressed(sf::Mouse::Butt
on::Left)) {
                float targetY =
orderItems[draggingIndex].rect.getPosition()
.y;
                int newPos = 0;
                for (size_t i = 0; i <
orderItems.size(); ++i)
                    if (static_cast<int>(i)
!= draggingIndex && targetY >
orderItems[i].rect.getPosition().y)
newPos++;
                OrderItem item =
orderItems[draggingIndex];
```

```
orderItems.erase(orderItems.begin() +
draggingIndex);

orderItems.insert(orderItems.begin() +
newPos, item);
                for (size_t i = 0; i <
orderItems.size(); ++i) {
                    orderItems[i].currentPos
= static_cast<int>(i);

orderItems[i].rect.setPosition({
(WINDOW_WIDTH - 600) / 2.f, questionDisplay-
>getPosition().y + 60 + questionDisplay-
>getLocalBounds().size.y + 20 +
static_cast<float>(i) * 50 });
                    sf::FloatRect tb =
orderItems[i].text.getLocalBounds();

orderItems[i].text.setPosition({
orderItems[i].rect.getPosition().x + 10,
orderItems[i].rect.getPosition().y + (40 -
tb.size.y) / 2.f - tb.position.y });
                }
                draggingIndex = -1;
            }
        }

        // Анимация полёта осколка
        if (fragmentFlying) {
            flyProgress =
flyClock.getElapsedTime().asSeconds() /
flyDurationAnim;
            if (flyProgress >= 1.f) {
                fragmentFlying = false;
                proceedNext();
            }
            else {
                flyingFragment-
>setPosition(flyStartPos + (flyTargetPos -
flyStartPos) * flyProgress);
            }
        }

        // Анимация текста на пред-
инструктаже
        if (currentState == PRE_INSTRUCTION)
{
            if (!textAnimationStarted) {
                animatedText->reset();
                textAnimationStarted = true;
            }
            animatedText->update();
        }
        else {
            if (textSound->getStatus() ==
sf::SoundSource::Status::Playing) {
                textSound->stop();
            }
        }

        // Затемнение
        if (fading) {
            float elapsed =
fadeClock.getElapsedTime().asSeconds();
            float alpha = fadeIn ? (elapsed
/ fadeDurationSec) * 255 : 255 - (elapsed /
fadeDurationSec) * 255;
            if (alpha > 255) alpha = 255;
            if (alpha < 0) alpha = 0;
            fadeRect-
>setFillColor(sf::Color(0, 0, 0,
static_cast<unsigned char>(alpha)));
            if (elapsed >= fadeDurationSec)
{
                fading = false;
                currentState =
nextStateAfterFade;
                if (currentState ==
PRE_INSTRUCTION) {
                    if
(!textAnimationStarted) {
                        animatedText-
>reset();
                        textAnimationStarted
= true;
                    }
                }
                if (currentState == MENU) {
                    music->play();
                }
                fadeRect-
>setFillColor(sf::Color(0, 0, 0, 0));
            }
        }

        // ---- Hover-эффект для кнопок ----
        auto applyHover = [&](sf::Sprite*
btn, sf::Text* txt = nullptr, float
scaleFactor = 1.1f) {
            if (btn) {
                float originalScale =
originalScales[btn];
                if
(isPointInSpriteWithPadding(*btn, mousePos,
5.f)) {
                    btn->setScale({
originalScale * scaleFactor, originalScale *
scaleFactor });
                    if (txt) {
                        unsigned int
baseSize = (txt == menuStartText.get() ? 32
:
                                    txt ==
menuExitText.get() ? 32 :
                                    txt ==
menuSettingsText.get() ? 28 :
                                    txt ==
startButtonText.get() ? 30 :
                                    txt ==
checkText.get() ? 20 :
                                    txt ==
nextText.get() ? 20 :
                                    txt ==
hintText.get() ? 22 :
                                    txt ==
backFromCrystalText.get() ? 24 :
                                    txt ==
backToMenuText.get() ? 20 :
                                    txt ==
aboutText.get() ? 20 :
                                    txt ==
settingsBackText.get() ? 20 :
                                    txt ==
musicBtn1Text.get() ? 24 :
                                    txt ==
musicBtn2Text.get() ? 24 :
                                    txt ==
musicBtn3Text.get() ? 24 :
                                    txt ==
crystalViewText.get() ? 14 :
                                    txt ==
finalStartText.get() ? 32 :
                                    txt ==
saveBtnText.get() ? 28 : 20);
                        unsigned int newSize
= static_cast<unsigned int>(baseSize *
scaleFactor);
                        txt-
>setCharacterSize(newSize);

centerTextOnButton(*txt, *btn);
                    }
                }
                else {
                    btn->setScale({
originalScale, originalScale });
                    if (txt) {
```

```
                        unsigned int
baseSize = (txt == menuStartText.get() ? 32
:
                                txt ==
menuExitText.get() ? 32 :
                                txt ==
menuSettingsText.get() ? 28 :
                                txt ==
startButtonText.get() ? 30 :
                                txt ==
checkText.get() ? 20 :
                                txt ==
nextText.get() ? 20 :
                                txt ==
hintText.get() ? 22 :
                                txt ==
backFromCrystalText.get() ? 24 :
                                txt ==
backToMenuText.get() ? 20 :
                                txt ==
aboutText.get() ? 20 :
                                txt ==
settingsBackText.get() ? 20 :
                                txt ==
musicBtn1Text.get() ? 24 :
                                txt ==
musicBtn2Text.get() ? 24 :
                                txt ==
musicBtn3Text.get() ? 24 :
                                txt ==
crystalViewText.get() ? 14 :
                                txt ==
finalStartText.get() ? 32 :
                                txt ==
saveBtnText.get() ? 28 : 20);
                            txt-
>setCharacterSize(baseSize);

centerTextOnButton(*txt, *btn);
                    }
                }
            }
            };

        // Применяем hover
        if (currentState == MENU) {
            applyHover(menuStartBtn.get(),
menuStartText.get());
            applyHover(menuExitBtn.get(),
menuExitText.get());

applyHover(menuSettingsBtn.get(),
menuSettingsText.get());
        }
        else if (currentState ==
PRE_INSTRUCTION) {
            applyHover(startButton.get(),
startButtonText.get());
            applyHover(topSettingsBtn.get(),
nullptr);
        }
        else if (currentState == QUIZ) {
            if (viewingCrystal) {

applyHover(backFromCrystalBtn.get(),
backFromCrystalText.get());

applyHover(topSettingsBtn.get(), nullptr);
            }
            else {

applyHover(crystalViewBtn.get(),
crystalViewText.get());

applyHover(topSettingsBtn.get(), nullptr);
                if (!isLastQuestion ||
!answerChecked) {

applyHover(checkButton.get(),
checkText.get());
                    const Question& q =
currentQuestions[currentQuestionIdx];
                    if (q.type ==
QuestionType::FIND_ERROR && hintsLeft > 0) {

applyHover(hintButton.get(),
hintText.get());
                    }
                }
                applyHover(nextButton.get(),
nextText.get());

applyHover(backToMenuBtn.get(),
backToMenuText.get());
            }
        }
        else if (currentState == SETTINGS) {
            applyHover(aboutBtn.get(),
aboutText.get());

applyHover(settingsBackBtn.get(),
settingsBackText.get());
            applyHover(musicBtn1.get(),
musicBtn1Text.get());
            applyHover(musicBtn2.get(),
musicBtn2Text.get());
            applyHover(musicBtn3.get(),
musicBtn3Text.get());
        }
        else if (currentState == FINAL) {
            applyHover(finalStartBtn.get(),
finalStartText.get());
            applyHover(saveBtn.get(),
saveBtnText.get());
        }

        // ---- Отрисовка ----
        window.clear();
        if (currentState == MENU) {
            window.draw(*background);
            sf::Text title(*font, L"Массивы
в языке C++", 48);

title.setFillColor(sf::Color(200, 200, 200,
180));
            title.setPosition({ WINDOW_WIDTH
/ 2.f - title.getLocalBounds().size.x / 2.f,
100.f });
            window.draw(title);
            window.draw(*menuStartBtn);
            window.draw(*menuStartText);
            window.draw(*menuExitBtn);
            window.draw(*menuExitText);
            window.draw(*menuSettingsBtn);
            window.draw(*menuSettingsText);
        }
        else if (currentState ==
PRE_INSTRUCTION) {
            window.draw(*background);
            window.draw(*crystalContour);
            window.draw(*textBg);
            animatedText->draw(window);
            window.draw(*startButton);
            window.draw(*startButtonText);
            window.draw(*topSettingsBtn);
        }
        else if (currentState == QUIZ) {
            if (viewingCrystal) {
                window.draw(*background);

window.draw(*crystalContour);
                for (int i = 0; i < 10; ++i)
if (fragmentsCollected[i])
window.draw(*fragmentSprites[i]);

window.draw(*backFromCrystalBtn);

window.draw(*backFromCrystalText);

window.draw(*topSettingsBtn);
```

```
            }
            else {

window.draw(*gameBackground);

window.draw(*crystalViewBtn);

window.draw(*crystalViewText);

window.draw(*topSettingsBtn);
                if (!isLastQuestion ||
!answerChecked) {

window.draw(*checkButton);
                    window.draw(*checkText);
                }
                window.draw(*nextButton);
                window.draw(*nextText);
                window.draw(*backToMenuBtn);

window.draw(*backToMenuText);
                window.draw(*taskNumber);

window.draw(*questionDisplay);

                const Question& q =
currentQuestions[currentQuestionIdx];
                if (q.type ==
QuestionType::CHOICE) {
                    for (auto& p :
optionPanels) { window.draw(p.rect);
window.draw(p.text); }
                }
                else if (q.type ==
QuestionType::FILL_CODE) {

window.draw(*codeBackground);
                    for (auto& txt :
codeLineTexts) window.draw(txt);
                    window.draw(*inputBox);
                    window.draw(*inputText);
                }
                else if (q.type ==
QuestionType::FIND_ERROR) {

window.draw(*codeBackground);
                    for (int i = 0; i <
static_cast<int>(codeLineTexts.size()); ++i)
{

window.draw(codeLineTexts[i]);
                        if (editing &&
editingLine == i && cursorVisible) {
                            sf::Text
cursorText(*font, "|", 16);

cursorText.setFillColor(sf::Color::White);
                            std::string
leftPart = editedLineContent.substr(0,
cursorPos);
                            sf::Text
leftText(*font, utf8_to_sfstring(leftPart),
16);
                            float xPos =
codeLineTexts[i].getPosition().x +
leftText.getLocalBounds().size.x;
                            float yPos =
codeLineTexts[i].getPosition().y;

cursorText.setPosition({ xPos, yPos });

window.draw(cursorText);
                        }
                    }
                    if (!isLastQuestion ||
!answerChecked) {
                        if (hintsLeft > 0) {

window.draw(*hintButton);

window.draw(*hintText);
                        }
                    }
                    if (hintVisible) {

window.draw(*hintOverlayBg);

window.draw(*hintOverlayText);
                    }
                }
                else if (q.type ==
QuestionType::ORDER) {
                    for (auto& oi :
orderItems) { window.draw(oi.rect);
window.draw(oi.text); }
                }

                if (answerChecked &&
!correctAnswerText->getString().isEmpty())

window.draw(*correctAnswerText);
                if (fragmentFlying)
window.draw(*flyingFragment);
                if (flashEffect) {
                    sf::RectangleShape
flash({ static_cast<float>(WINDOW_WIDTH),
static_cast<float>(WINDOW_HEIGHT) });

flash.setFillColor(sf::Color(255, 255, 255,
150));
                    window.draw(flash);
                }
                if (!resultMessage.empty()
&& !fragmentFlying) {
                    sf::Text msg(*font,
utf8_to_sfstring(resultMessage), 26);

msg.setFillColor(sf::Color::Yellow);
                    msg.setPosition({
WINDOW_WIDTH / 2 -
msg.getLocalBounds().size.x / 2,
WINDOW_HEIGHT - 80 });
                    window.draw(msg);
                }
            }
        }
        else if (currentState == SETTINGS) {
            window.draw(*background);
            sf::Text settingsTitle(*font,
L"Настройки", 40);

settingsTitle.setFillColor(sf::Color::White)
;
            settingsTitle.setPosition({
WINDOW_WIDTH / 2.f -
settingsTitle.getLocalBounds().size.x / 2.f,
100.f });
            window.draw(settingsTitle);

            if (aboutAuthor) {
                std::string info = "ОБ
АВТОРЕ\nавтором является Караченко София
Денисовна,\nучащаяся группы ПЗТ-45
Гродненского
государственного\nполитехнического
колледжа.\nПрограмма написана на языке
C++\nс использованием библиотеки
SFML.\nСреда разработки Microsoft visual
studio.\n\nГродно, 2026 г.";
                sf::Text aboutText(*font,
utf8_to_sfstring(info), 20);

aboutText.setFillColor(sf::Color::White);
                sf::FloatRect bounds =
aboutText.getLocalBounds();
                aboutText.setPosition({
WINDOW_WIDTH / 2.f - bounds.size.x / 2.f,
200.f });
                window.draw(aboutText);
```

```
window.draw(*settingsBackBtn);

window.draw(*settingsBackText);
            }
            else {
                // Ползунок громкости
                window.draw(*sliderBg);
                window.draw(*sliderFill);
                window.draw(*sliderHandle);
                window.draw(*volumeLabel);

                // Музыка
                window.draw(*musicLabel);
                window.draw(*musicBtn1);
                window.draw(*musicBtn1Text);
                window.draw(*musicBtn2);
                window.draw(*musicBtn2Text);
                window.draw(*musicBtn3);
                window.draw(*musicBtn3Text);

                window.draw(*aboutBtn);
                window.draw(*aboutText);

window.draw(*settingsBackBtn);

window.draw(*settingsBackText);
            }
        }
        else if (currentState == FINAL) {
            window.draw(*gameBackground);
            if (!finalCrystalComplete) {
                if (resultLoaded)
resultSound->play();
                if (!resultSoundPlayed) {
                    if (resultEndLoaded)
resultEndSound->play();
                    resultSoundPlayed =
true;
                }
                finalCrystalComplete = true;
            }

            sf::Sprite
finalCrystal(*crystalContour);

finalCrystal.setPosition(crystalPos +
GLOBAL_OFFSET);
            if (score >= 8) {
                finalCrystal.setScale({
crystalScale, crystalScale });

finalCrystal.setColor(sf::Color(255, 255,
200, 255));
            }
            else if (score >= 5) {
                finalCrystal.setScale({
crystalScale, crystalScale });

finalCrystal.setColor(sf::Color(180, 180,
150, 255));
            }
            else {
                finalCrystal.setScale({
crystalScale, crystalScale });

finalCrystal.setColor(sf::Color(100, 100,
100, 255));
            }
            window.draw(finalCrystal);
            for (int i = 0; i < score && i <
10; ++i) {
                sf::Sprite
frag(*fragmentSprites[i]);
                frag.setScale({
fragmentScale, fragmentScale });

frag.setPosition(fragmentSprites[i]-
>getPosition());
                window.draw(frag);
            }

            // ---- Финальный текст с фоном
----
            std::string resultText;
            if (score >= 9) {
                resultText = "👑 [9-10] —
Абсолютный резонанас. Мастер\n"
                    "Статус: макс. запас
кристаллов (9-10 шт) — Сверхгенерация!\n"
                    "Анализ: идеальное
прохождение. Все фрагменты кода
интерпретированы эталонно. Вопросы с
подвохом решены без ошибок.\n"
                    "Рекомендация: модуль
исчерпан. Энергосистема под вашим
контролем.";
            }
            else if (score >= 7) {
                resultText = "🚀 [7-8] —
Высокоэффективный синтез\n"
                    "Статус: отличный улов
(7-8 кристаллов).\n"
                    "Анализ: высокий уровень
анализа кода. Алгоритмы сортировки и
фильтрации выполнены верно. Минимальные
потери в вопросах по указателям.\n"
                    "Рекомендация: вы готовы
к сложным архитектурам. Изучите связь
статической памяти и адресов.";
            }
            else if (score >= 4) {
                resultText = "⚡ [4-6] —
Стабильная добыча\n"
                    "Статус: собрано
умеренное кол-во кристаллов (4-6 шт).\n"
                    "Анализ: базовые задания
по одномерным массивам выполнены, но в
многомерных матрицах и sizeof возникли
пустоты. Часть ресурсов утеряна из-за выхода
за границы.\n"
                    "Рекомендация: закрепите
вложенные циклы и передачу массивов в
функции.";
            }
            else {
                resultText = "💥 [1-3] —
Критический сбой энергосети\n"
                    "Статус: кристаллы не
получены (0-3 шт).\n"
                    "Анализ: перегрузка
энергосети. Критические ошибки в синтаксисе
и индексации. Неверный расчет циклов
разрушил структуру данных.\n"
                    "Рекомендация: повторите
объявление массивов и правила индексации (с
нуля).";
            }

            std::string wrappedFinal =
wrapText(resultText, *font, 18, WINDOW_WIDTH
- 200);
            sf::Text finalMsg(*font,
utf8_to_sfstring(wrappedFinal), 18);

finalMsg.setFillColor(sf::Color::White);

            sf::FloatRect msgBounds =
finalMsg.getLocalBounds();
            sf::RectangleShape
finalBg(sf::Vector2f{ msgBounds.size.x + 40,
msgBounds.size.y + 40 });

finalBg.setFillColor(sf::Color(0, 0, 0,
180));

finalBg.setOutlineThickness(2.f);

finalBg.setOutlineColor(sf::Color::White);
```

```
            finalBg.setPosition({
(WINDOW_WIDTH - finalBg.getSize().x) / 2.f,
110.f });
            finalMsg.setPosition({
finalBg.getPosition().x + 20,
finalBg.getPosition().y + 20 });

            window.draw(finalBg);
            window.draw(finalMsg);

            sf::Text scoreText(*font,
utf8_to_sfstring("Ваш результат: " +
std::to_string(score) + "/10"), 30);

scoreText.setFillColor(sf::Color(180, 0,
0));
            scoreText.setPosition({
WINDOW_WIDTH / 2.f -
scoreText.getLocalBounds().size.x / 2.f,
50.f });
            window.draw(scoreText);

            window.draw(*saveBtn);
            window.draw(*saveBtnText);
            window.draw(*finalStartBtn);
            window.draw(*finalStartText);

            if (!resultMessage.empty() &&
resultMessage.find("Сохранено") !=
std::string::npos) {
                sf::Text msg(*font,
utf8_to_sfstring(resultMessage), 20);

msg.setFillColor(sf::Color::Green);
                msg.setPosition({
WINDOW_WIDTH / 2.f -
msg.getLocalBounds().size.x / 2.f,
WINDOW_HEIGHT - 180.f });
                window.draw(msg);
            }
            else if (!resultMessage.empty()
&& resultMessage.find("Ошибка") !=
std::string::npos) {
                sf::Text msg(*font,
utf8_to_sfstring(resultMessage), 20);

msg.setFillColor(sf::Color::Red);
                msg.setPosition({
WINDOW_WIDTH / 2.f -
msg.getLocalBounds().size.x / 2.f,
WINDOW_HEIGHT - 180.f });
                window.draw(msg);
            }
        }

        if (fading) window.draw(*fadeRect);
        window.display();
    }
    return 0;
}
```

Приложение А
Листинг программы